Evaluating the Cybersecurity Risk of Real-world, Machine Learning Production Systems

RON BITTON and NADAV MAMAN, Ben-Gurion University of the Negev, Israel
INDERJEET SINGH and SATORU MOMIYAMA, NEC, Japan
YUVAL ELOVICI and ASAF SHABTAI, Ben-Gurion University of the Negev, Israel

Although cyberattacks on machine learning (ML) production systems can be harmful, today, security practitioners are ill-equipped, lacking methodologies and tactical tools that would allow them to analyze the security risks of their ML-based systems. In this article, we perform a comprehensive threat analysis of ML production systems. In this analysis, we follow the ontology presented by NIST for evaluating enterprise network security risk and apply it to ML-based production systems. Specifically, we (1) enumerate the assets of a typical ML production system, (2) describe the threat model (i.e., potential adversaries, their capabilities, and their main goal), (3) identify the various threats to ML systems, and (4) review a large number of attacks, demonstrated in previous studies, which can realize these threats. To quantify the risk posed by adversarial machine learning (AML) threat, we introduce a novel scoring system that assigns a severity score to different AML attacks. The proposed scoring system utilizes the analytic hierarchy process (AHP) for ranking—with the assistance of security experts—various attributes of the attacks. Finally, we developed an extension to the MulVAL attack graph generation and analysis framework to incorporate cyberattacks on ML production systems. Using this extension, security practitioners can apply attack graph analysis methods in environments that include ML components thus providing security practitioners with a methodological and practical tool for both evaluating the impact and quantifying the risk of a cyberattack targeting ML production systems.

CCS Concepts: • **Security and privacy;** • **Computing methodologies** → **Machine learning;**

Additional Key Words and Phrases: Adversarial machine learning, attack graphs, threat analysis, risk assessment

ACM Reference format:
Ron Bitton, Nadav Maman, Inderjeet Singh, Satoru Momiyama, Yuval Elovici, and Asaf Shabtai. 2023. Evaluating the Cybersecurity Risk of Real-world, Machine Learning Production Systems. *ACM Comput. Surv.* 55, 9, Article 183 (January 2023), 36 pages.
https://doi.org/10.1145/3559104

1 INTRODUCTION

In the past few years, we have witnessed a revolution in the use and deployment of **artificial intelligence (AI)** and **machine learning (ML)** by organizations and large enterprises. Gartner's 2019 CIO Survey shows that 37% of enterprises have implemented some form of ML, which represents

Authors' addresses: R. Bitton, N. Maman, Y. Elovici and A. Shabtai (corresponding author), Ben-Gurion University of the Negev, Beer-Sheba, Israel; emails: ronbit@post.bgu.ac.il, nadavmam@post.bgu.ac.il, elovici@bgu.ac.il, shabtaia@bgu.ac.il; I. Singh and S. Momiyama, NEC, Japan; emails: inderjeet@nec.com, satoru-momiyama@nec.com.

0360-0300/2023/01-ART183 $15.00
https://doi.org/10.1145/3559104

a 270% increase in four years.[1] Furthermore, among Fortune 1000 companies, 91.5% of businesses have reported ongoing investment in ML-based technologies.[2] ML algorithms are used for a variety of purposes, including cybersecurity [12, 13, 54, 74, 77, 84], fraud detection [6, 66, 81], financial trading [29, 59], disease prediction in healthcare [21, 60, 71], and autonomous driving [22, 45]. In most of these use cases, critical decisions are made based on the processing and output of the ML algorithm, thus a failure in these systems could have significant impact on business continuity, revenue, and, in some use cases, even human lives.

1.1 The Security of Machine Learning Systems

The increasing popularity of ML systems, coupled with their critical role in the decision-making process, makes them an ideal target for attackers. Unfortunately, ML algorithms are not vulnerability-free. Recent studies have demonstrated that ML algorithms suffer from a special type of logical vulnerabilities stemming from inherent limitations of the underlying ML algorithms. To exploit these vulnerabilities, attackers utilize an attack technique referred to as **adversarial machine learning (AML)**, which has already been demonstrated in security and ML research [15, 35, 80].

Despite the fact that a cyberattack on ML systems can cause harm to systems, enterprises, and the people dependent on them, industry practitioners today are not equipped with the tactical and strategic tools needed to analyze, detect, protect, and respond to cyberattacks on their ML systems [43].

Specifically, current risk assessment tools suffer from two main limitations. First, traditional risk assessment tools model the target system using raw telemetry information such as the system's network topology, installed software (and their known vulnerabilities), running services, and file permissions, however, the business logic of these applications or services is not considered. When evaluating the security of machine learning applications business logic is extremely important. For instance, the impact of manipulating an arbitrary file is not the same as manipulating a model's training data, since by manipulating a model's training data, the attacker can influence the model's decisions, and by doing so, the attacker can extend the scope of the attack beyond the specific host storing the training file. Second, current risk assessment tools are aimed at modeling software and network vulnerabilities and accordingly consider traditional attack techniques. However, ML systems suffer from logical vulnerabilities, which cannot be exploited via traditional attack techniques. Therefore, to model attacks on ML systems, the risk assessment tool must also consider a new class of attacks known as adversarial machine learning attacks.

1.2 Scope and Purpose

Recent studies on AML have mainly focused on developing frameworks and libraries for evaluating the robustness of ML models to AML. Notable frameworks are the CleverHans adversarial examples library [61], the **Adversarial Robustness Toolbox (ART)** [56], the Foolbox toolbox [67], the SecML library [51], and MLsploit [23]. Although these frameworks, which implement various AML attack techniques, can be used to quantify the resilience of an ML model to different kinds of AML attack methods, they focus solely on implementing algorithms aimed at finding adversarial examples and measuring their performance against the target ML model. These frameworks are not designed to quantify the risk of ML-based system to cybersecurity threats in general and to AML threats in particular, since they ignore the specific deployment of the target environment,

[1] https://www.gartner.com/en/newsroom/press-releases/2019-01-21-gartner-survey-shows-37-percent-of-organizations-have.
[2] http://newvantage.com/wp-content/uploads/2020/01/NewVantage-Partners-Big-Data-and-AI-Executive-Survey-2020-1.pdf.

as well as the different attributes of the attack technique (e.g., attacker model, attack impact, and attack performance), which must be considered in practical risk assessment.

In this article, we take a significant step toward securing ML production systems by integrating these unique systems and their vulnerabilities into cybersecurity risk assessment frameworks. We begin by exploring ML production systems and performing a comprehensive threat analysis, based on the ontology presented by NIST for evaluating enterprise network security risk. Specifically, we enumerate the main components/assets of ML production systems, describe the threat model (i.e., potential adversaries, their capabilities, and their main goal), identify the threats to ML production systems, and review a large number of attacks, demonstrated in previous studies, which can realize the threats.

In addition, to quantify the risk posed by AML threat, we introduce a novel scoring system that assigns a severity score to different AML attacks. The proposed scoring system utilizes the **analytic hierarchy process (AHP)** for ranking [68] with the assistance of security experts, various attributes of the attacks.

We also extend the MulVAL attack graph generation and analysis framework to incorporate cyberattacks on ML production systems. This extension considers both traditional attack techniques as well as AML attacks. We conclude by demonstrating practical uses of the proposed extension, which will ultimately provide security practitioners with a tactical and practical tool for both evaluating the impact and quantifying the risk of a cyberattack targeting ML production systems.

1.3 Contribution

The main contributions of this research can be summarized as follows: First, we conducted a comprehensive threat analysis of ML production systems. Second, we develop the **Common Adversarial Machine Learning Vulnerability Scoring System (CMLVSS)** for assigning a severity score to AML attacks. Third, we develop an extension to the MulVAL attack graph generation and analysis framework that provides the ability to consider cyberattacks on ML production systems.

2 ML PRODUCTION SYSTEMS

ML systems are used for various use cases, each of which may have different deployment requirements and constraints. In general, ML pipelines can be classified into two types: research pipelines and production pipelines (see Figure 1). Understanding the characteristics of these two different pipelines is crucial for evaluating the risk of ML systems. In this section, we discuss the main differences between a typical research pipeline and a typical production pipeline. Our analysis is based on two reference environments: Michelangelo—Uber's ML platform[3]—and TensorFlow Extended—Google's ML production platform [8]. The main characteristics and components of research and production pipelines are summarized in Table 1.

2.1 The Characteristics of a Research Pipeline

In a research pipeline, the main objectives are to explore novel ML applications that can benefit business goals and quickly develop proofs of concept for such applications. Therefore, in a research pipeline, the ML researcher (or data scientist) mainly focuses on extracting and analyzing data, defining new features, and testing (evaluating) different ML algorithms. The interactive nature of these tasks, coupled with the requirement to test new concepts and algorithms quickly, results in the following characteristics of a typical ML research pipeline:

[3]https://eng.uber.com/michelangelo-machine-learning-platform/.

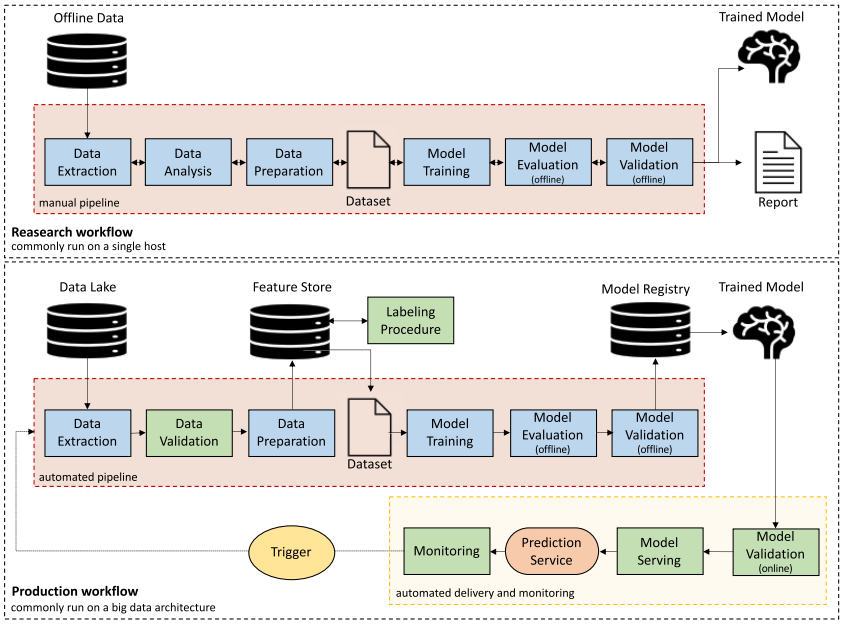

Fig. 1. Typical ML research and production workflows.

Table 1. A Comparison of the Main Characteristics and Components of Machine Learning Research and Production Workflows

	Data	Deployment	Orchestration	Pipeline								Delivery				
				Data Extraction	Data Analysis	Data Validation	Data Preparation	Model Training	Model Offline Evaluation	Model Offline Validation	Model Online Validation	Continuous Model Retraining	Performance Monitoring	Feature Store	Model Registry	Model Serving
Research	Offline	Single Host	Manual	●	●	○	●	●	●	●	○	○	○	○	○	○
Production	Online	Cluster	Automated	●	○	●	●	●	●	●	●	●	●	●	●	●

(i) **Data.** In most cases, the data scientist works with static, offline datasets. This way, the data scientist can explore the data and test new features quickly, using simple scripts, without the overhead of writing production-level code that connects to the enterprise's big data architecture components.

(ii) **Deployment.** The entire process (including data extraction, data analysis, data preparation, model training, and model evaluation) is implemented on a single host.

(iii) **Delivery.** The main outputs are a trained model and a report describing the application and evaluation results. The data scientist provides these outputs to the engineering team who serves the model as a prediction service in the production environment, i.e., the research pipeline does not include components required for using a predication model as a service. Retraining the model with new data and actively monitoring model performance are tasks that are rarely considered in a research setup.

(iv) **Orchestration.** Every component (including data interpretation, data preparation, model training, and validation) is executed manually using dedicated scripts. In addition, the transition from one step to another is performed manually.

2.2 The Characteristics of a Production Pipeline

When ML models need to be deployed in production, engineering requirements (such as security, availability, throughput, scalability, compliance with privacy and bias regulations, etc.) must also be considered. Furthermore, when working with online real-world data, data trends often change with time. This behavior, known as concept drift, must be considered to ensure high performance over time. Therefore, when ML needs to be deployed in production, the actual pipeline becomes much more complex:

(i) **Data.** Large-scale live data must be utilized efficiently. Therefore, an ML production pipeline must be integrated into the enterprise's big data architectures (i.e., a cluster of computers).
(ii) **Deployment.** Data volumes can be tremendous. Therefore, a production pipeline is commonly deployed on big data architecture.
(iii) **Delivery.** When working with online real-world data, data trends often change with time. Therefore, model performance must be actively monitored to detect performance degradation. To mitigate such degradation, the ML models must be retrained frequently using fresh data. This way, the ML model can learn emerging patterns and trends over time. However, before retraining a model, the training data must be validated to detect changes in the statistical properties of the data and identify data anomalies, which can cause the model to learn incorrect concepts. Furthermore, the retrained model must be re-evaluated (and validated) to ensure high performance. A production pipeline also includes components to store and manage models and features (a model repository and feature store).
(iv) **Orchestration.** Since models change frequently in a production pipeline, model management must be automated and cannot be handled manually by a data scientist. This includes the execution of each step in the pipeline, as well as the transitions between steps.

3 THE SECURITY OF ML SYSTEMS

In this section, we analyze the security of ML production systems. Our analysis is conducted according to a threat analysis ontology presented in Figure 2, which is based on the NIST ontology for evaluating enterprise security risk. In Table 2, we provide a brief description of the various entities presented in the proposed ontology. Within the proposed threat analysis, we begin by enumerating the assets of a typical ML production system; identifying these assets is a crucial step in threat modeling. Then, we describe the threat model, which outlines the potential adversaries, their capabilities, and their main goals. Next, we describe the various threats to ML production systems. Finally, we enumerate the attacks that can realize these threats.

3.1 Target Assets

A typical ML production system includes the following main components:

[A1] Data Lake. This component is a big data architecture that is responsible for storing and processing large volumes of data.

[A2] Data Extraction. This component is responsible for selecting and integrating data from the various data sources that exist in the data lake.

[A3] Data Validation. This component receives fresh data extracted from the data lake and is responsible for validating the input data to prevent the model from learning incorrect concepts.

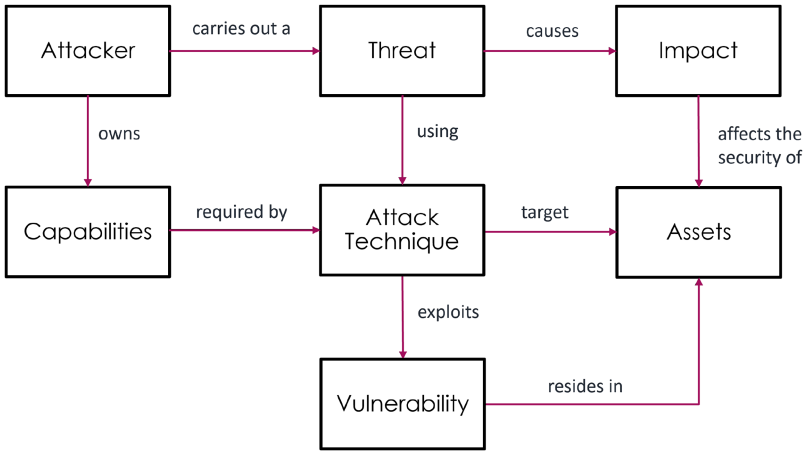

Fig. 2. Threat analysis ontology.

Table 2. Brief Description of the Entities Presented in the Proposed Ontology

Entity	Description
Assets	The data, components, processes, and services that comprise a ML production pipeline.
Vulnerability	A characteristic of an asset or a technology that makes it prone to an attack.
Attacker	An individual, group, or state responsible for an event or incident that impacts, or has the potential to impact, the security or safety of the system. Threat actors may have different capabilities and resources and may perform different attacks.
Capabilities	Refers to the capabilities that are available to the attacker. Within the context of AML attackers, we distinguish between access capabilities and knowledge capabilities.
Impact	By exploiting vulnerabilities, threats are able to have security impact on the vulnerable assets, e.g., violate the confidentiality of the data used for training a model or tamper with the ML model's integrity. Security impact may lead to operational impact.
Threat	A potential violation of a security property, such as integrity, confidentiality, or availability. In a threat, an attacker exploits a vulnerability to perform an attack.
Attack Technique	An act or method that violates the security policy of a system.

[A4] Data Preparation. This component receives validated data and is responsible for preparing the data for the ML task. The preparation includes data cleaning, data transformations, and feature engineering. In addition, this component is responsible for splitting the data into training, validation, and test sets.

[A5] Feature Store. This component is a data warehouse responsible for storing and logging features for ML tasks. The main benefits of using this component are the ability to reuse available feature sets in different ML tasks; serve up-to-date feature values; and avoid training-serving skew by using the feature store as the data source for experimentation, continuous training, and online serving.

[A6] Data Labeling. This component (or process) is responsible for tagging data samples. The process can be manual but is usually performed or assisted by software. The input for this process is commonly raw data and predefined labeling logic, and the output consists of the labels of each data sample.

[A7] Feature Selection. This component is responsible for selecting the subset of features used to train the model.

[A8] Model Training. This component is responsible for training ML algorithms with prepared data. The output of this step is a trained model.

[A9] Model. The trained model file, which is produced by feeding training data to the learning algorithm.

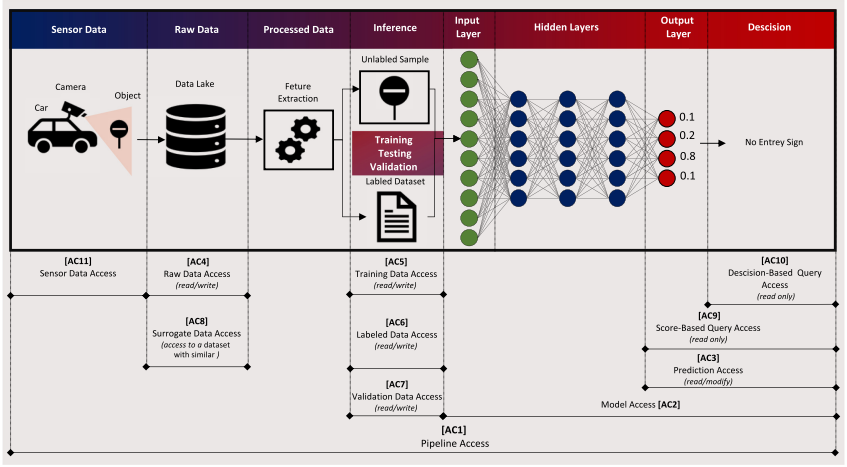

Fig. 3. An illustration of the various access capabilities available to the adversary for different threat models.

[A10] Model Repository. This component is a database responsible for storing models, their performance, versioning, and other configuration information (e.g., features used, hyperparameter values).

[A11] Model Evaluation (offline). This component is responsible for evaluating (using various measurements) the performance of a trained model on an unseen offline dataset (known as a test set).

[A12] Model Validation (offline). This component is responsible for validating that the new model is adequate for deployment (i.e., is better than a certain baseline).

[A13] Model Validation (online). This component (also known as A/B testing) is responsible for evaluating the performance of a trained model on a small portion of unseen online data. The performance of the existing and new models are then compared, and the superior model is selected for serving.

[A14] Model Serving. This component is responsible for deploying the model to provide predictions (e.g., using a REST API).

[A15] Model Performance Monitoring. This component is responsible for actively monitoring model performance to detect performance degradation.

3.2 Threat Model

In this section, we define the threat model, which is based on the attacker's access capabilities and knowledge regarding the target system.

3.2.1 Attacker's Access Capabilities. The set of capabilities accessible to the attacker regarding the target system (see Figure 3).

[AC1] Pipeline Access. In this threat model, we assume a complete access of the adversary to all of the components included in the target pipeline (including the sensor data, raw data, training data, algorithm, hyperparameters, model, and classification).

[AC2] Model Access. In this threat model, we assume an adversary that has access to the exact model used in the target pipeline.

[AC3] Prediction Access. In this threat model, we assume an adversary that can modify the output (predictions) of the model used in the target pipeline.

[AC4] Raw Data Access. In this threat model, we assume an adversary with access to the raw data (this data will be used for training the model after applying some data transformations).

[AC5] Training Data Access. In this threat model, we assume an adversary with access to the dataset used for training the target model.

[AC6] Labeled Data Access. In this threat model, we assume an adversary with access to the labels of the dataset used for training the target model (i.e., the adversary is able to manipulate the labels of training instances).

[AC7] Validation Data Access. In this threat model, we assume an adversary with access to the dataset used for the external validation of the target model.

[AC8] Surrogate Data Access. In this threat model, we assume an adversary that has access to a reference dataset with similar characteristics and data distribution to the dataset used to train the model.

[AC9] Score-based Query Access. In this threat model, we assume an adversary with the ability to query the trained model. We further assume that given a query, the attacker has access to the probability vector (i.e., score), which describes the confidence for each class. However, we do not assume the adversary has access to the training data.

[AC10] Decision-based Query Access. In this threat model, we assume an adversary with the ability to query the trained model. We further assume that given a query, the attacker only has access to the decision. However, we do not assume the adversary has access to the probability vector or the training data.

[AC11] Sensor Data Access. In this threat model, we assume an adversary with the ability to manipulate the data captured/measured by the sensors (e.g., an adversary that can manipulate an object that resides within a camera's line of sight).

3.2.2 Attacker's Knowledge. Information regarding the target system that is available to the attacker and can be used to more effectively generate the attacks.

[AK1] Perfect Knowledge. In this threat model, we assume an adversary with complete knowledge on the target pipeline, including the training data, algorithm, hyperparameters, model, and classification.

[AK2] Model Knowledge. In this threat model, we assume an adversary that knows the exact model used in the target pipeline (e.g., the target pipeline uses a public model).

[AK3] Hyperparameter Knowledge. In this threat model, we assume an adversary that knows the exact algorithm and hyperparameters used to train the algorithm (e.g., for artificial neural networks, the hyperparameters include the network architecture, number of epochs used to train the model, learning rate).

[AK4] Algorithm Knowledge. In this threat model, we assume an adversary that knows the algorithm used to train the model but does not know the exact hyperparameters.

[AK5] Training Data Knowledge. In this threat model, we assume an adversary that knows all/part of the training data used for training the model (including the exact feature transformations applied on the raw data).

[AK6] Raw Data Knowledge. In this threat model, we assume an adversary that knows the exact data used for training the model (e.g., when a target model is trained on a public dataset) but is not aware of the exact feature transformations applied to the raw data.

[AK8] Data Property Knowledge. In this threat model, we assume an adversary that knows the statistical properties of the data used to train the model.

[AK9] Task Knowledge. In this threat model, we assume an adversary that has general knowledge on the ML task, including the general type of inputs and outputs (e.g., the attacker knows that the pipeline is used to extract points of interest from the user's historical geolocations).

3.2.3 Attacker's Goal (or Impact). Indicates what type of security impact (violation) the attacker wants to achieve.

[AG1] Tampering. This threat category is associated with malicious activities that would compromise the *integrity* of the ML system.

[AG2] Denial of Service (DoS). This threat category is associated with malicious activities that would compromise the *availability* of the ML system. This goal can be achieved by causing the model to make a significant number of incorrect predictions, rendering the model unusable, or by sending specially crafted examples that increase the model's prediction time and/or cause high latency.

[AG3] Information Disclosure. This threat category is associated with malicious activities that would compromise the *privacy* of the ML system.

3.3 Threat Categories

In this section, we enumerate the main threats (by category) to ML production systems.

[T1] Evading an ML System. An adversary that causes the ML system to provide incorrect outputs for a specific input, for instance, causing a malware detection classifier to classify a malicious file as benign.

[T2] ML Model Corruption. An adversary that causes the ML system to deploy a corrupted model, for instance, causing the ML system to deploy a corrupted malware detection classifier that includes a backdoor that will classify specific malicious files as benign.

[T3] Membership Inference. An adversary that causes the ML system to leak information regarding the existence of a given input sample in the training set.

[T4] Data Property Inference. An adversary that causes the ML system to leak information in such a way that general properties of the training dataset (e.g., feature distribution, input types) are exposed.

[T5] Data Reconstruction (theft). An adversary that causes the ML system to leak information in such a way that some of the data samples used to train the target model are exposed.

[T6] Model Extraction. An adversary that causes the ML system to leak information in such a way that the model used in the target pipeline is exposed.

[T7] Resource Exhaustion. An adversary that causes the the ML model to use more resources at inference time. This threat compromises the *availability* of the machine learning model.

4 AML ATTACK TECHNIQUES

In this section, we review various AML attack techniques against ML systems. For each attack technique, we provide a brief description of the attack method, identify the assets targeted by the adversary, and list the relevant threat model. Table 3 summarizes the results derived from the proposed threat

4.1 Evading an ML System

In evasion attacks the adversary exploits the ML model *[A9]* by generating a crafted input sample (an adversarial example) that is very similar to some other correctly classified input but is incorrectly classified by the ML model. The adversary's main objective is to compromise the integrity *[AG1]* of the ML model *[A9]* by causing the ML system to provide incorrect outputs for a specific input *[T1]*. Evasion attacks can be broadly classified based on the following criteria: attack technique, which characterizes the technique used by the attacker to craft the adversarial example (e.g., gradient-based, boundary-based, and transferability-based); the threat model, which characterizes the attacker's access capabilities and attacker's knowledge on the ML-based system (e.g.,

Table 3. Adversarial Machine Learning Threat Analysis

Threat Category	Attack Technique	Target Assets	Threat Model: Access Capabilities: Pipeline Access [AC1]	Model Access [AC2]	Prediction Access [AC3]	Raw Data Access [AC4]	Training Data Access [AC5]	Labeled Data Access [AC6]	Validation Data Access [AC7]	Surrogate Data Access [AC8]	Score-Based Query Access [AC9]	Decision-Based Query Access [AC10]	Threat Model: System Knowledge: Perfect Knowledge [AK1]	Model Knowledge [AK2]	Hyperparameter Knowledge [AK3]	Algorithm Knowledge [AK4]	Training Data Knowledge [AK5]	Raw Data Knowledge [AK6]	Data Property Knowledge [AK7]	Task Knowledge [AK8]	Attack Impact: Tampering [AG1]	Denial of Service [AG2]	Information Disclosure [AG3]	S
Evading an ML system *[T1]*	[AT1] Gradient-based white-box evasion attacks (such as C&W [15]) FGSM [32], and BIM [44])	[A9] Model	○	○	○	○	○	○	○	○	○	⊙	●	●	●	●	●	●	●	●	●	○	○	6.6
	[AT2] Boundary-based black-box (score-based) evasion attacks (such as ZOO [18])	[A9] Model	○	○	○	○	○	○	○	○	●	⊙	○	○	○	○	○	○	○	●	●	○	○	7.5
	[AT3] Boundary-based black-box (decision-based) evasion attacks (such as HopSkipJump [17])	[A9] Model	○	○	○	○	○	○	○	○	○	●	○	○	○	○	○	○	○	●	●	○	○	8
	[AT4] Transferability-based black-box (decision-based) evasion attacks that utilize reference data (such as Jacobian Data Augmentation [63])	[A9] Model	○	○	○	○	○	○	○	⊙	○	●	○	○	○	○	○	○	●	●	●	○	○	8
	[AT5] Transferability-based black-box (decision-based) evasion attacks that utilize training data (such as [63])	[A9] Model	○	○	○	○	○	○	○	○	○	●	○	○	○	○	●	●	●	●	●	○	○	7.2
	[AT6] Gradient-based targeted white-box poisoning attack (such as [38, 42, 51])	[A9] Model	○	○	○	⊙	⊙	⊙	○	○	○	⊙	●	●	●	●	●	●	●	●	●	○	○	5.1
	[AT7] Transferability-based targeted black-box poisoning attack (error-specific or error-generic such as [9, 38, 54]))	[A9] Model	○	○	○	⊙	⊙	⊙	○	●	○	⊙	○	○	○	○	●	○	●	●	●	○	○	6.5
	[AT8] Gradient-based targeted white-box poisoning attack against feature selection (such as [84])	[A7] Feature selection	○	○	○	⊙	⊙	⊙	○	○	○	⊙	●	●	●	●	●	●	●	●	●	○	○	5.1
	[AT9] Transferability-based targeted black-box poisoning attack against feature selection (such as [84])	[A7] Feature selection	○	○	○	⊙	⊙	⊙	○	●	○	⊙	○	○	○	○	●	○	●	●	●	○	○	6.5
ML model corruption *[T2]*	[AT10] Gradient-based indiscriminate white-box poisoning attack (such as [38, 42, 51])	[A9] Model	○	○	○	⊙	⊙	⊙	○	○	○	⊙	●	●	●	●	●	●	●	●	○	●	○	4.5
	[AT11] Transferability-based indiscriminate black-box poisoning attack (such as [9, 38, 54])	[A9] Model	○	○	○	⊙	⊙	⊙	○	●	○	⊙	○	○	○	○	●	○	●	●	○	●	○	5.9
	[AT12] Gradient-based indiscriminate white-box poisoning attack against feature selection (such as [84])	[A7] Feature selection	○	○	○	⊙	⊙	⊙	○	○	○	⊙	●	●	●	●	●	●	●	●	○	●	○	4.5
	[AT13] Transferability-based indiscriminate black-box poisoning attack against feature selection (such as [84])	[A7] Feature selection	○	○	○	⊙	⊙	⊙	○	●	○	⊙	○	○	○	○	●	○	●	●	○	●	○	5.9
Membership inference *[T3]*	[AT14] Shadow training-based black-box membership inference attacks [73]	[A1] Data lake [A5] Feature store	○	○	○	⊙	⊙	○	○	○	○	●	●	○	○	○	○	⊙	○	○	●	○	○	●	7.8
	[AT15] Gradient-based white-box membership inference attacks (such as [56])	[A1] Data lake [A5] Feature store	○	●	●	⊙	⊙	○	○	○	○	●	○	○	●	●	●	⊙	○	○	●	○	○	●	6.6
Data property inference *[T4]*	[AT16] Shadow training-based property inference attacks [5, 28, 76]	[A1] Data lake [A5] Feature store	○	○	○	⊙	⊙	○	○	○	○	●	●	○	○	○	○	⊙	○	○	⊙	○	○	●	7.8
Data reconstruction *[T5]*	[AT17] Maximum a posteriori-based gray-box data reconstruction attacks [26, 34]	[A1] Data lake [A5] Feature store	○	⊙	⊙	⊙	⊙	○	○	○	○	●	●	○	○	○	⊙	⊙	○	⊙	○	○	●	6.8
	[AT18] Gradient optimization-based gray-box data reconstruction attacks [25]	[A1] Data lake [A5] Feature store	○	⊙	⊙	⊙	⊙	○	○	○	○	●	●	○	○	○	⊙	⊙	○	⊙	○	○	●	6.8
	[AT19] Black-box data reconstruction attacks [86]	[A1] Data lake [A5] Feature store	○	○	○	⊙	⊙	○	○	○	○	●	●	○	○	○	○	⊙	⊙	○	⊙	○	○	●	7.5
Model extraction *[T6]*	[AT20] Query-based black-box model extraction attacks [16, 39, 58, 64]	[A9] Model	○	○	○	○	○	○	○	○	○	●	●	○	○	○	○	⊙	⊙	○	⊙	○	○	●	8.1
Resource exhaustion *[T5]*	[AT21] Gradient-based resource exhaustion attacks [71, 74]	[A9] Model	○	○	○	○	○	○	○	○	○	⊙	●	●	●	●	●	●	●	●	○	●	○	5.9
	[AT22] Query-based resource exhaustion attacks (interactive black-box) [74]	[A1] Data lake [A5] Feature store	○	○	○	○	○	○	○	○	●	⊙	○	○	○	○	○	○	○	●	○	●	○	6.9
	[AT23] Transferability-based resource exhaustion attacks (complete black-box) [74]	[A1] Data lake [A5] Feature store	○	○	○	○	○	○	○	●	○	⊙	○	○	○	○	●	○	●	●	○	●	○	5.9

Access capabilities - ○: does not require this type of access, ⊙: limited access of this type is required, ●: complete access of this type is required.
System knowledge - ○: does not require this type of knowledge, ⊙: requires partial knowledge, ●: requires full knowledge.
Attack goal - ○: The attacker can achieve this type of goal by executing this attack, ●: The attacker cannot achieve this type of goal by executing this attack.
Mapping the Various Attack Techniques to their Threat Categories, Target Assets, Required Threat Model (in Terms of Access Capabilities and Knowledge Capabilities), Security Impact and Severity Score (as Derived by Security Experts using the AHP).

white-box, black-box); and the attack's specificity, which characterizes the goal of the attacker (e.g., targeted vs. untargeted).

[AT1] Gradient-based white-box evasion attacks (targeted and untargeted). In these types of attacks, the adversary manipulates an input sample in such a way that the classification loss is maximized. The specific manipulation is determined by calculating the gradients of the classification loss with respect to the input sample [15, 20, 32, 44, 52, 64]. For example, in the FGSM attack [32], the adversarial example is calculated by adding noise (in the direction of the gradient of the classification loss with respect to the input sample) to the input sample. Since these types of attacks are based on gradient computation, they consider a white-box adversary with perfect knowledge on the target pipeline *[AK1]*. In addition, they further assume that the adversary has the ability to query the trained model *[AC9][AC10]*; this requirement is necessary for executing the attack (i.e., sending the adversarial example crafted for classification).

[AT2] Boundary-based black-box (score-based) evasion attacks (targeted and untargeted). Similar to *[AT1]*, these types of attacks generate the adversarial example based on the gradients of the classification loss with respect to the input sample, however, in contrast to *[AT1]*, which *calculate* the gradients, these attacks *estimate* the gradients [18, 36]. For example, in the ZOO attack [18], the adversary utilizes zeroth-order optimization methods to directly estimate the gradients of the target model. Since these attacks are not based on gradient computation, they consider an adversary with general knowledge on the task *[AK8]* and black-box score-based query access *[AC9]*.

[AT3] Boundary-based black-box (decision-based) evasion attacks (targeted and untargeted). Similar to *[AT2]*, these types of attacks also *estimate* the gradients. However, in contrast to *[AT2]*, which estimate the gradient by using the classification score (i.e., vector of probabilities), these attacks estimate the gradients using the label alone (without using the vector of probabilities) [14, 17]. For example, in the HopSkipJump attack [17], the adversary utilizes Monte Carlo methods to directly estimate the gradients of the target model. As a result, these attacks consider an adversary with general knowledge on the task *[AK8]* and black-box decision-based query access *[AC10]*.

[AT4] Transferability-based black-box (decision-based) evasion attacks that utilize reference data (targeted and untargeted). These types of attacks include the following three main phases: First, the adversary creates a surrogate model by training a learning algorithm on a reference dataset with similar characteristics and distribution as the training set *[AC8]*. Second, the adversary generates adversarial examples by executing white-box gradient-based attacks *[AT1]* on the surrogate model. Third, the adversary uses the adversarial examples (generated against the surrogate model) to attack the target model. These attacks exploit the *transferability* property of ML models [32, 62, 80], i.e., adversarial examples that affect one model can often affect other models, even if they trained using different learning algorithms, hyperparameters, or training sets, as long as all models were trained to perform the same task (e.g., image classification). For example in Reference [62], the adversary utilizes Jacobian data augmentation to generate a surrogate dataset given an initial substitute training set of limited size. Since the gradient-based attacks are executed on a surrogate model, they can be executed by a black-box attacker with very limited query access *[AC10]* as long as the adversary knows the general task *[AK8]* and data properties *[AK7]* and has access to a reference dataset *[AC8]*.

[AT5] Transferability-based black-box (decision-based) evasion attacks (targeted and untargeted) that utilize training data. Similar to *[AT4]*, the adversary executes gradient-based attacks on a surrogate model (created by the attacker). The only difference is that in these attacks, the adversary creates a surrogate model by training a learning algorithm on the exact dataset used to train the target model rather than on a reference dataset. Therefore, as long as the adversary

knows the training data used to train the model *[AK5][AK6][AK7]* and has knowledge on the general task *[AK8]*, these attacks can be executed by a black-box attacker with very limited query access *[AC10]*.

4.2 Poisoning Attacks

In poisoning attacks, the adversary exploits the ML training procedure *[A8]* by injecting crafted data samples into the dataset used for training the learning algorithm *[AG2]*. The adversary's main objective is either to compromise the availability *[AG2]* of the ML model *[A9]* by causing the ML system to deploy a corrupted model *[T2]* or to compromise the integrity *[AG1]* of the ML model *[A9]* by causing the ML system to deploy a model that provides incorrect outputs for a specific input *[T1]*. Poisoning attacks can be classified based on the following criteria: attack technique, which characterizes the technique used by the attacker to craft the malicious input samples (i.e., gradient-based, if the attack is based on a gradient optimization method; and transferability-based, if the attack utilizes the transferability property of ML models); threat model, which characterizes the attacker's access capabilities and the attacker's knowledge regarding the system (e.g., white-box and black-box); attack's specificity, which characterizes the goal of the attacker (i.e., targeted, if the attack aims to cause misclassification of a specific set of samples; or indiscriminate, if the attack aims to cause misclassification of any sample); and error specificity, which characterizes the type of error in multiclass problems (i.e., error-specific, if the attacker aims to have a sample misclassified as a specific class; or error-generic, if the attacker aims to have a sample misclassified as any of the classes other than the true class).

Poisoning attacks that are used for evading an ML system are also known as backdoor and Trojaning attacks [33, 48, 86].

[AT6] Gradient-based targeted white-box poisoning attacks (error-specific and error-generic). In these types of attacks, the adversary manipulates a small number of training samples in such a way that the classification loss of some other set of samples targeted by the attacker is maximized *[T1][AG1]*. The specific manipulation is calculated by solving—using gradient-optimization techniques—a bilevel optimization problem where the outer optimization aims at manipulating the malicious input to maximize the loss function on a dataset that includes the samples targeted by the attacker, while the inner optimization corresponds to retraining the learning algorithm on a dataset that includes the malicious examples [33, 38, 42, 48, 50, 86]. Since these types of attacks are based on the adversary's ability to manipulate a small number of training samples, the adversary must have the ability to write to the dataset used to train the model *[AC3],[AC4],[AC5]*. In addition, since these attacks are based on gradient computation, the adversary must possess perfect knowledge on the target pipeline (including the model architecture and parameters) *[AK1]*. Furthermore, to execute the attack, the adversary must have the ability to query the trained model *[AC10]*.

[AT7] Transferability-based targeted black-box poisoning attacks (error-specific and error-generic). Similar to *[AT6]*, the adversary manipulates a small number of training samples in such a way that the classification loss of some other set of samples targeted by the attacker is maximized *[T1],[AG1]*. Therefore, these types of attacks also require the adversary to have the ability to write to the dataset used to train the model *[AC3],[AC4],[AC5]*. However, in contrast to *[AT6]*, which calculate the gradients using the target model (which requires perfect knowledge on the system), these attacks calculate the gradients using a surrogate model [9, 38, 53] (created by training a learning algorithm on a reference dataset whose characteristics and distribution are similar to the training set) *[AC8]*. As a result, they can be executed by a black-box attacker with very limited query access *[AC10]*, as long as the adversary knows the general task *[AK8]* and data properties [AK7] and has access to a reference dataset *[AC8]*.

[AT8] Gradient-based targeted white-box poisoning attacks against feature selection (error-specific and error-generic). In these types of attacks, the adversary manipulates a small number of training samples in a way that leads the feature selection algorithm to select a specific subset of features targeted by the attacker *[T1],[AG1]*. The specific manipulation is calculated by solving—using gradient-optimization techniques—a bilevel optimization problem where the outer optimization aims at manipulating the malicious input to maximize the feature selection loss function on a dataset that includes the samples targeted by the attacker, while the inner optimization corresponds to retraining the feature selection algorithm on a dataset that includes the malicious examples [83]. Since these types of attacks are based on the adversary's ability to manipulate a small number of training samples, the adversary must have the ability to write to the dataset used for training the model *[AC3],[AC4],[AC5]*. In addition, since these attacks are based on gradient computation, the adversary must possess perfect knowledge on the target pipeline (including the model architecture and parameters) *[AK1]*. Furthermore, to execute the attack, the adversary must have the ability to query the trained model *[AC10]*.

[AT9] Transferability-based targeted black-box poisoning attacks against feature selection (error-specific and error-generic). Similar to *[AT6]*, the adversary manipulates a small number of training samples in a way that leads the feature selection algorithm to select a specific subset of features targeted by the attacker *[T1],[AG1]*. Therefore, these types of attacks also require the adversary to have the ability to write to the dataset used to train the model *[AC3],[AC4],[AC5]*. However, in contrast to *[AT6]*, which calculate the gradients using the target model (which requires perfect knowledge on the system), these attacks calculate the gradients using a surrogate model [83] *[AC8]*. As a result, they can be executed by a black-box attacker with very limited query access *[AC10]*, as long as the adversary knows the general task *[AK8]* and data properties [AK7] and has access to reference dataset *[AC8]*.

4.3 ML Model Corruption

In this class of poisoning attacks the adversary's main objective is to compromise the availability *[AG2]* of the ML model *[A9]* by causing the ML system to deploy a corrupted model; i.e., a model that misclassifies any input sample (indiscriminately).

[AT10] Gradient-based indiscriminate white-box poisoning attacks (error-specific and error-generic). In these types of attacks, the adversary manipulates a small number of training samples in a such a way that the classification loss of an untainted dataset is maximized *[T1],[AG2]*. The specific manipulation is calculated by solving—using gradient-optimization techniques—a bilevel optimization problem where the outer optimization aims at manipulating the malicious input to maximize the loss function on a dataset that includes the samples targeted by the attacker, while the inner optimization corresponds to retraining the learning algorithm on an untainted dataset (e.g., a validation dataset that does not include malicious examples) [38, 42, 50]. Since these types of attacks are based on the adversary's ability to manipulate a small number of training samples, the adversary must have the ability to write to the dataset used to train the model *[AC3],[AC4],[AC5]*. In addition, since these attacks are based on gradient computation, the adversary must possess perfect knowledge on the target pipeline (including the model architecture and parameters) *[AK1]*. Furthermore, to execute the attack, the adversary must have the ability to query the trained model *[AC10]*.

[AT11] Transferability-based indiscriminate black-box poisoning attacks (error-specific and error-generic). Similar to *[AT10]*, the adversary manipulates a small number of training samples in a such a way that the classification loss of an untainted dataset is maximized *[T1],[AG2]*. Therefore, these types of attacks also require the adversary to have the ability to write to the dataset used to train the model *[AC3],[AC4],[AC5]*. However, in contrast to *[AT10]*, which

calculate the gradients using the target model (which requires perfect knowledge on the system), these attacks calculate the gradients using a surrogate model [9, 38, 53] (created by training a learning algorithm on a reference dataset whose characteristics and distribution are similar to the training set) *[AC8]*. As a result, they can be executed by a black-box attacker with very limited query access *[AC10]*, as long as the adversary knows the general task *[AK8]* and data properties [AK7] and has access to reference dataset *[AC8]*.

[AT12] Gradient-based indiscriminate white-box poisoning attacks against feature selection (error-specific and error-generic). In these types of attacks, the adversary manipulates a small number of training samples in a way that leads the feature selection algorithm to select a different subset of features *[T1],[AG2]*. The specific manipulation is calculated by solving—using gradient-optimization techniques—a bilevel optimization problem where the outer optimization aims at manipulating the malicious input to maximize the feature selection loss function on a dataset that includes the samples targeted by the attacker, while the inner optimization corresponds to retraining the feature selection algorithm on an untainted dataset [83]. Since these types of attacks are based on the adversary's ability to manipulate a small number of training samples, the adversary must have the ability to write to the dataset used to train the model *[AC3],[AC4],[AC5]*. In addition, since these attacks are based on gradient computation, the adversary must possess perfect knowledge on the target pipeline (including the model architecture and parameters) *[AK1]*. Furthermore, to execute the attack, the adversary must have the ability to query the trained model *[AC10]*.

[AT13] Transferability-based indiscriminate black-box poisoning attacks against feature selection (error-specific and error-generic). Similar to *[AT6]*, the adversary manipulates a small number of training samples in a way that leads the feature selection algorithm to select a specific subset of features targeted by the attacker *[T1]*. Therefore, these types of attacks also require the adversary to have the ability to write to the dataset used to train the model *[AC3],[AC4],[AC5]*. However, in contrast to *[AT10]*, which calculate the gradients using the target model (which requires perfect knowledge on the system), transferability-based poisoning attacks calculate the gradients using a surrogate model (created by running the feature selection algorithm on a reference dataset whose characteristics and distribution are similar to the training set) *[AC8]*. As a result, they can be executed by a black-box attacker with very limited query access *[AC10]*, as long as the adversary knows the general task *[AK8]* and data properties [AK7] and has access to reference dataset *[AC8]*.

4.4 Membership Inference

In membership inference attacks the adversary exploits the ML model *[A9]* to expose private information and thereby compromise the privacy *[AG3]* of the ML model *[A9]* or the data used to train it *[A1],[A5]*.

[AT14] Shadow training-based black-box membership inference attacks. In these types of attacks, given an ML model *[A9]* and a record, the adversary's goal is to determine whether the record was used as part of the model's training dataset *[T3]* and thereby compromise the privacy of that record *[AG3]*. These attacks exploit the fact that ML models often behave differently on the data that they were trained on than they behave on the test data [72]. Specifically, as introduced in Reference [72], these attacks utilize shadow models that only require access to the prediction vector of the target ML system. However, these attacks assume the adversary has partial knowledge about the target system's training data by exploiting either partial access to raw *[AC4]* and training data *[AC4]* or by having prior partial knowledge on the training data *[AK5]*.

[AT15] Gradient-based white-box membership inference attacks. In these types of attacks, the adversary has partial knowledge on the training data *[AK5]*, *[AC5]*, and *[AC6]* and

complete access *[AC2],[AC3]* and knowledge *[AK2],[AK3]*, *[AK4]* on the target system's model. Membership inference *[T3]* is performed in these types of attacks, based on gradient behavior observed during the training phase [55].

4.5 Data Property Inference

Similar to membership inference attacks, in data property inference the adversary exploits the ML model *[A9]* to expose private information and thereby compromise the privacy *[AG3]* of the ML model *[A9]* or the data used to train it *[A1],[A5]*.

[AT16] Shadow training-based property inference attacks. In these types of attacks, the adversary extracts information about the features that are not correlated with the learning task *[T4]*. These attacks can be performed using shadow models, leveraging the adversary's access to the target model's predictions (*[AC9][AC10]*), and partial knowledge on the training data, leveraging *[AC4]*, *[AC5]*, *[AK5]*, and the target system's task *[AK8]* [5, 28, 75].

4.6 Data Reconstruction

[AT17] Maximum *a posteriori*-based gray-box data reconstruction attacks. In these types of attacks [26, 34] falling under *[T5]*, an adversary reconstructs training samples, and their respective labels *[A1], [A5]*, of the target system's ML model [A9]. The adversary uses a **maximum a posteriori (MAP)** estimate of the attribute that maximizes the probability of observing the known parameters, while assuming partial access *[AC2], [AC3], [AC4], [AC5], [AC9], [AC10]* and knowledge *[AK5], [AK6], [AK8]* of the target model *[A9]* and feature information from the training data.

[AT18] Gradient optimization-based gray-box data reconstruction attacks. In these attacks [25], an adversary has model knowledge *[AK2]*, task knowledge *[AK8]*, and partial training data knowledge *[AK5]*. In addition, the adversary can obtain the required knowledge by exploiting *[AC2]*, *[AC3]*, *[AC4]*, and *[AC5]*. These attacks solve an optimization problem, using gradient descent in the input sample space to recover the input data point *[T5]*.

[AT19] Black-box data reconstruction attacks. In these types of attacks [85], an adversary reconstructs training samples *[T5]* while having limited knowldge on training task *[AK8]* and only query access *[AC9]* and *[AC10]* to the target model. These attacks use an autoencoder setting where the target model plays the role of an encoder, and the trainable decoder network tries to reconstruct the training sample on the prediction vector *[AC9]* or *[AC10]* of the target model.

4.7 Model Extraction

[AT20] Query-based black-box model extraction attacks. In these types of attacks [16, 39, 57, 63], an adversary tries to extract complete knowledge on a target ML model *[A9]* by fully reconstructing it or creating a substitute model that behaves very similarly *[T6]*. These attacks only require query access *[AC9],[AC10]* to the target model and limited knowledge on the training task *[AK4],[AK5],[AK8]*.

4.8 Resource Exhaustion

[AT21] Gradient-based resource exhaustion attacks (white-box). In these types of attacks [70, 73], the adversary searches (using gradient optimization techniques) for inputs that increase activation values of the model across all of the layers simultaneously. High activation values prevent hardware optimization and therefore increase latency. Since these types of attacks are based on gradient computation, they consider perfect knowledge of the target pipeline *[AK1]*. In addition, they further assume that the adversary has the ability to query the trained model *[AC10]*; this requirement is only necessary for executing the attack (i.e., sending the crafted adversarial example for classification).

[AT22] Query-based resource exhaustion attacks (interactive black-box). In these types of attacks [73], the adversary searches (using a genetic algorithm) for inputs that increase the model's latency. In contrast to the gradient-based method, this method does not require access to the model's parameters. Instead, it requires query access *[AC10]* to the target model.

[AT23] Transferability-based resource exhaustion attacks (complete black-box) In this type of attack [73], the adversary creates a surrogate model and searches (using a genetic algorithm or gradient-based optimization) for inputs that increase the model's latency. Since gradient-based optimization techniques are executed on a surrogate model, they can be executed by a black-box attacker with very limited query access *[AC10]* as long as the adversary knows the general task *[AK8]*, data properties [AK7], or has access to a reference dataset *[AK6]*.

5 COMMON ADVERSARIAL MACHINE LEARNING VULNERABILITY SCORING SYSTEM (CMLVSS)

As demonstrated in Section 3, AML attack techniques may require a different threat model and commonly have a different security impact. Thus, it is possible that different AML attack techniques do not have the same severity level. Unfortunately, recent studies on AML have not provided methods for quantifying the severity of AML attacks. The ability to integrate AML threats into traditional cybersecurity risk assessment tools depends on the development of such a method.

In this section, we present a risk assessment method for assigning a severity score to AML attacks, similar to the **common vulnerability scoring system (CVSS)**. We utilize the threat analysis described in Section 3 to identify the attributes of AML attacks that affect the attack's severity level. Then, we enlist the assistance of security experts to rank these attributes, and that ranking is used to derive the security severity score for each attack.

One of the first challenges faced when relying on people to rank multiple criteria, as needed in this case, is balancing the tradeoff between time complexity and accuracy. Previous studies have demonstrated that synchronous pairwise comparison between pairs of criteria resulted in a very accurate ranking, outperforming the approach of ranking multiple criteria simultaneously [68]. However, a pairwise comparison is not feasible when there is a large number of criteria. To address this challenge, we utilize a technique from the domain of group decision-making, the **analytic hierarchy process (AHP)** [68], which is specifically designed to overcome these drawbacks.

5.1 The Analytic Hierarchy Process (AHP)

The AHP is a comprehensive framework for quantifying the weights of decision criteria using a group of experts. Using the AHP, individual experts perform pairwise comparisons to estimate (using a specially designed questionnaire) the relative magnitudes of attributes concerning the decision criteria. However, in contrast to naive pairwise comparisons, the AHP decomposes the decision problem into a hierarchy of more easily comprehendible sub-problems, each of which can be analyzed independently. That is, in the AHP, experts only compare pairs of elements at the same level of the hierarchy, thus reducing the amount of comparison needed. Another advantage of the AHP is the ability to measure the internal consistency of each expert, as well as the agreement among a group of experts (we will discuss these measures in the sections that follow).

5.2 Using the AHP to Rank AML Attacks

The process of ranking AML attacks using the AHP includes the following six phases:

Phase 1: Decomposing the decision problem into a hierarchy of sub-problems. To utilize the AHP to rank AML attacks by their severity, we need to identify the attributes that affect the severity of an attack and organize them in a hierarchical structure. To do so, we follow the ontology

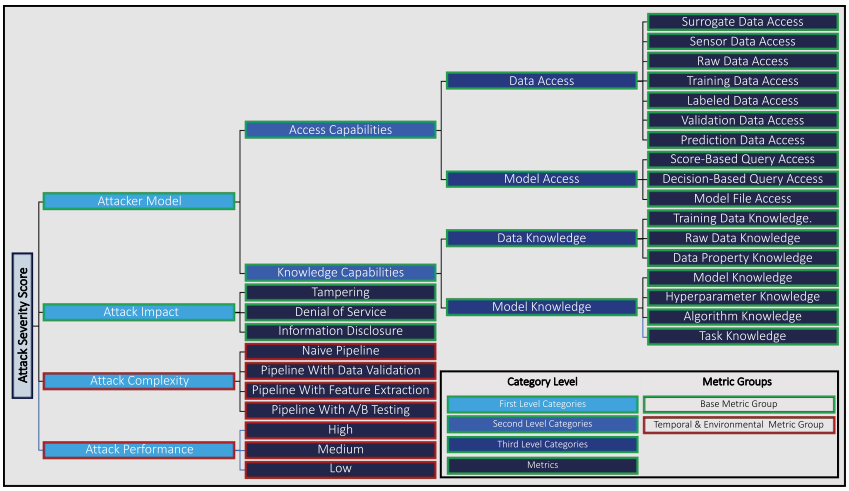

Fig. 4. AML attack severity taxonomy.

presented by the CVSS for the severity of traditional security vulnerabilities and modify it for the AML attack domain, based on the threat model described in Section 3.2.

The resulting taxonomy, which we refer to as the AML attack severity taxonomy, is presented in Figure 4. As can be seen, the proposed severity score is composed of four top-level categories: attacker model, attack impact, attack complexity, and attack performance. Note that similar to the CVSS, we distinguish between two types of attributes (denoted as metric groups): the base metric group, which represents the intrinsic qualities of an AML attack technique that are constant and do not depend on a specific attack implementation or a specific environmental configuration; and the temporal & environmental metric group, which represents the characteristics of an AML attack technique that are unique to a specific environmental configuration or attack implementation. Then, following the threat model described in Section 3.2, we define two second-level categories for the attacker model category: *attacker access capabilities* and *attacker knowledge capabilities*. Next, we define third-level categories to distinguish between *data access attributes* and *model access attributes*, as well as to distinguish between *data knowledge* and *model knowledge*. Finally, the leaves of the taxonomy (i.e., the basic attack attributes) are defined based on the threat model described in Section 3.2.

Phase 2: Constructing the questionnaire. In this step, we design a questionnaire that can be completed by security experts to rank pairs of attributes that are at the same level of the AML attack severity taxonomy. The proposed questionnaire was implemented in Excel and is provided as supplementary material. We also provide screenshots of the questionnaire in Figure 5. An example of a comparison question in the questionnaire is: *Which of the following two capabilities (denoted by "A" and "B") is more difficult for an attacker to obtain, and to what extent?* where A = *Surrogate Data Access*, and B = *Sensor Data Access*. The possible answers for every question are presented in a nine-level Likert scale (for importance ranking).

Phase 3: Calculating the weights of each attack attribute based on individual experts. In this step, the score for each element in the taxonomy is calculated according to the AHP methodology [68]. This is done for each ranker (i.e., expert) individually.

Phase 4: Validating internal consistency. In this step, we validate the internal consistency of each ranker. The validation is performed dynamically during ranking, using the consistency ratio

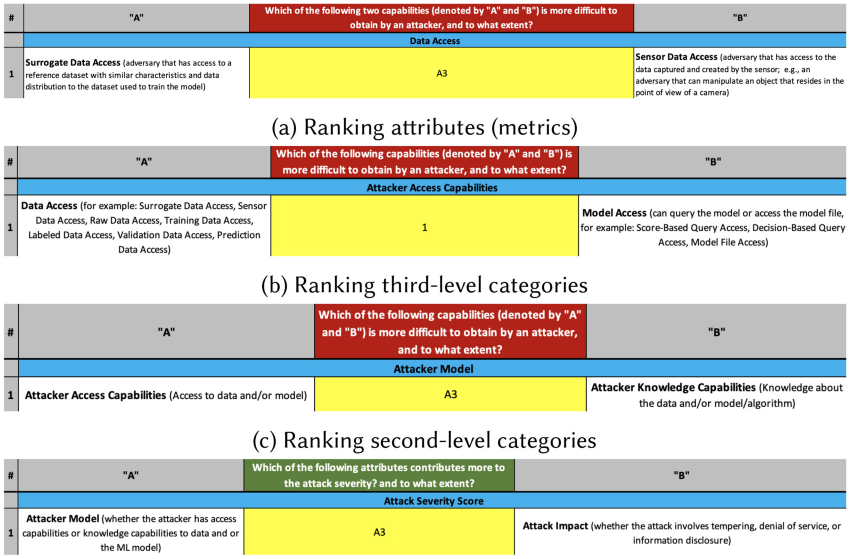

Fig. 5. Screenshots from the designed questionnaire.

measure (denoted by CR). According to the AHP, CR values that are less than 0.1 are considered consistent. Therefore, if the consistency ratio measure exceeds 0.1, then the experts were asked to revise their ranking to maintain consistency. By the end of this phase, all experts achieved CR values that are less than 0.1 for all comparison levels.

Phase 5: Testing external agreement. In this phase, we tested the external agreement among different experts. The evaluation was performed using Kendall's W test (i.e., Kendall's coefficient of concordance), which is a non-parametric statistical test commonly used to assess agreement among raters. Kendall's W ranges from zero (no agreement) to one (complete agreement), where a value greater than 0.6 is considered strong agreement.

Phase 6: Calculating the severity score of each attack. Finally, we compute the average score for each element in the taxonomy across all experts. At the end of this step, each element (including the leaves that represent the threat model criteria) are assigned a score that represents the contribution (weight) of each criterion to the impact of the corresponding AML attack. In this final step, given an AML attack, we can derive the severity score of the attack by combining (using a weighted average) the scores of all criteria that define the attack.

5.3 Results

We asked eight experts in the domains of AML and cybersecurity to complete the questionnaire. When testing the external agreement among the various experts, we achieved an agreement factor of 0.75, indicating strong agreement. Therefore, the scores of each element in the taxonomy were computed using the questionnaires of all raters.

In Figure 6, we present the importance of each metric to the attack severity level. Note that the importance of attributes that are associated with the attacker model or attack complexity categories quantifies the *negative* impact of the attribute on the severity of the attack. However, the importance of attributes that are associated with the attack impact and attack success rate indicates the *positive* impact of the attribute on the severity of the attack.

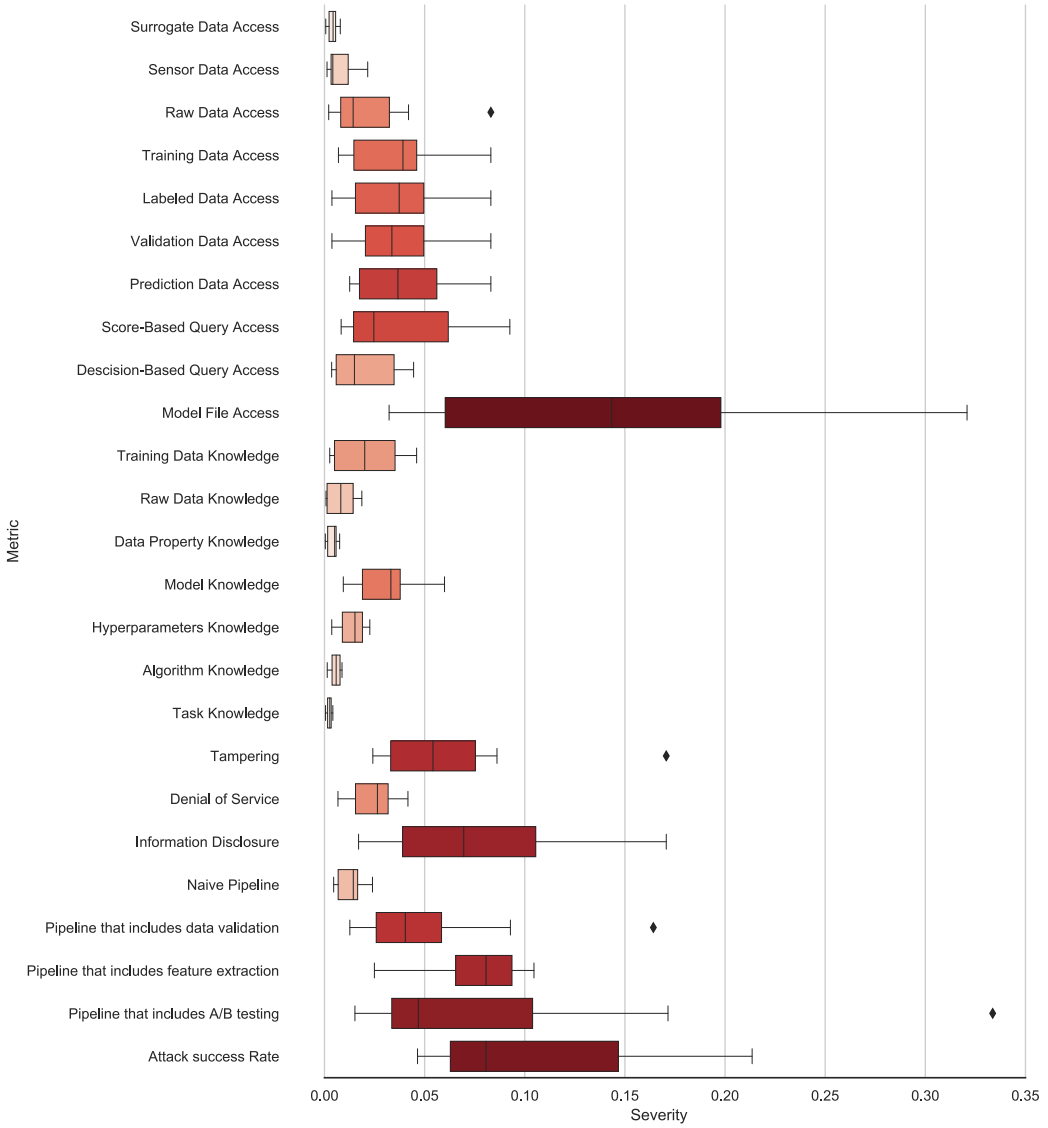

Fig. 6. Experts' ranking results: metric level.

As can be seen, in terms of the attacker model, access to the model file (that is currently deployed) is considered the capability that is most difficult for an attacker to obtain. In addition, it is considered to an attacker to obtain access to training data, labeled data, validation data, and prediction data. However, access to surrogate data or sensor data is considered easy for an attacker to obtain. Unsurprisingly, obtaining data/model knowledge is easier than obtaining data/model access.

Furthermore, the results show that attacks that resulted in tampering or information disclosure are more severe than attacks resulting in denial of service and that it is far more difficult to implement an attack on deployments that include A/B testing and feature extraction.

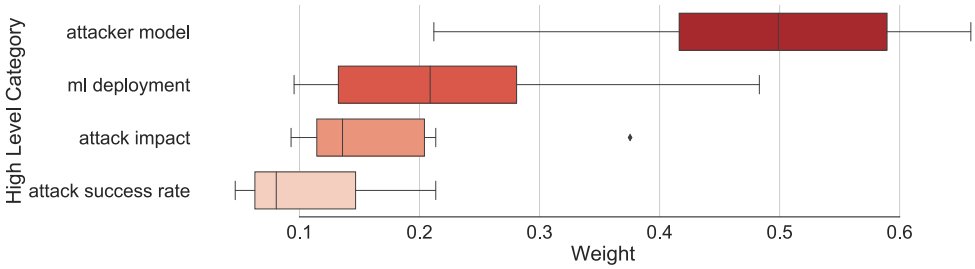

Fig. 7. Experts' ranking results: high-level category.

Table 4. The Importance of the High-level Categories on the Attack Severity (as Derived by Security Experts Using the AHP)

Threat Model																		Attack Impact			Environmental Metrics			
Access Capabilities											System Knowledge													
Pipeline Access [AC1]	Model Access [AC2]	Prediction Access [AC3]	Raw Data Access [AC4]	Training Data Access [AC5]	Labeled Data Access [AC6]	Validation Data Access [AC7]	Surrogate Data Access [AC8]	Score-Based Query Access [AC9]	Decision-Based Query Access [AC10]	Sensor Data Access [AC11]	Perfect Knowledge [AK1]	Model Knowledge [AK2]	Hyperparameter Knowledge [AK3]	Algorithm Knowledge [AK4]	Training Data Knowledge [AK5]	Raw Data Knowledge [AK6]	Data Property Knowledge [AK7]	Task Knowledge [AK8]	Tampering [AG1]	Denial of Service [AG2]	Information Disclosure [AG3]	Naive Pipeline	Pipeline with Data Validation	Pipeline with Feature Extraction	Pipeline with A/B Testing
Σ_{ac}	.144	.040	.025	.037	.038	.038	.004	.039	.020	.008	Σ_{sk}	.031	.014	.006	.022	.025	.004	.002	.066	.024	.081	.013	.055	.075	.095

In Figure 7 and Table 4, we present the importance of high-level categories (i.e., threat model, attack impact, and environmental metrics) on the attack severity. As can be seen, the attacker model is considered the most important aspect of attack severity, and the attack impact and attack success rate are less important.

In Figure 8, we visualize the risk landscape of the adversarial machine learning attack techniques based on the severity scores of the different attack techniques (which derived from the CMLVSS approach). The X-axis represents the likelihood for obtaining the required threat model. The Y-axis represents the security impact resulting from the attack. As can be seen, the most severe attacks are: black-box data reconstruction attacks, shadow training-based, black-box membership inference attacks, shadow training-based property inference attacks, and query-based, black-box model extraction attacks. We attribute this to the fact that (a) these attacks can operate in a complete black-box setting with minimal demands on the attacker; and (b) these attacks compromise the privacy of the model that was classified as the most severe security impact by the experts. However, the results show that gradient-based, indiscriminate, white-box poisoning attacks are less severe. We attribute that to the fact that these attacks require a very strong threat model (perfect knowledge of the target system). Furthermore, these attacks resulted in tampering, which was classified as the less severe security impact by the experts.

6 MULVAL EXTENSION

In the previous section (Section 5), we presented a traditional risk assessment process that is based on the **common vulnerability scoring system (CVSS)** [82]. This method is efficient (the overall risk in the system is evaluated very quickly) and widely used, however, it

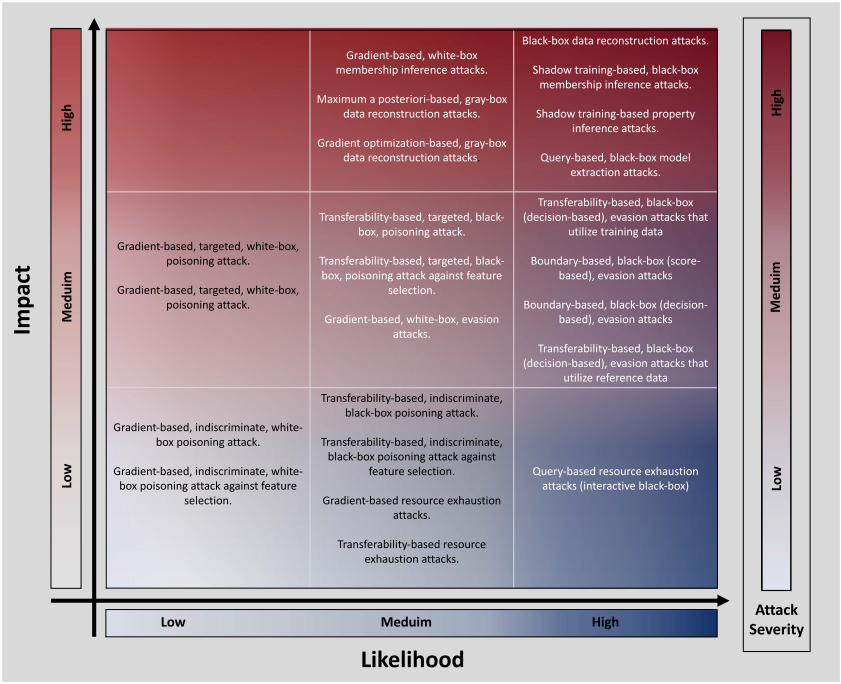

Fig. 8. Qualitative risk assessment analysis of adversarial machine learning attacks.

analyzes each vulnerability/attack technique individually and therefore ignores the prerequisites for exploiting a vulnerability, the system's topology, and the attacker's position and potential lateral movements.

An alternative risk assessment approach is based on logical attack graphs, e.g., References [19, 31, 40, 41, 69]. A logical attack graph is a structured representation of both the relationships between different vulnerabilities and the relationships between vulnerabilities and system assets; it can be thought of as a graphical representation of the attack surface, which can also represent the prerequisites and consequences of vulnerabilities and the attacker's potential lateral movements. The main benefits of risk assessment based on logical attack graphs are twofold: First, while traditional risk assessment methods analyze each vulnerability individually, attack-graph-based risk assessment also models the interactions between vulnerabilities and the lateral movements of the attacker. Second, while traditional risk assessment methods evaluate the severity of each vulnerability regardless of the specific environmental deployment and settings, attack-graph-based risk assessment considers the specific preconditions and impact of exploiting security vulnerabilities on the specific target environment. Thus, attack graphs enable very accurate and concrete risk assessment.

6.1 Introduction to Logical Attack Graphs

A logical attack graph is a directed graph that represents all possible attack scenarios. It specifies the relations between the specific system configuration (e.g., running services, installed software, and existing security vulnerabilities) and the attacker's potential privileges (e.g., executing arbitrary code, denial of service, and privilege escalation). To generate a logical attack graph, the consequence of each attack phase should be expressible as a propositional formula of the system

configurations. In logical attack graph terminology, propositional formulas are referred to as *interaction rules*, and system configurations (and attacker's privileges) are referred to as *predicates* (which can be either *primitive*, i.e., a basic standalone fact, or *derived*, i.e., complex facts that are derived based on other facts using an interaction rule).

Formally, a logical attack graph is defined as a tuple $AG = (N_r, N_p, N_d, E, \mathcal{L}, \mathcal{G})$, where:

- N_r - The set of derivation nodes. These nodes correspond to interaction rules and imply an *AND* relation between their incoming nodes.
- N_p - The set of nodes that represent primitive facts.
- N_d - The set of nodes that represent derived facts. These nodes imply an *OR* relation between their incoming nodes.
- $E \subseteq \{(N_p \cup N_d) \times N_r\} \cup \{N_r \times N_d\}$ - The set of edges.
- $\mathcal{L}$ - A mapping between nodes and their labels.
- $\mathcal{G}$ - The node that represents the attacker's goal.

6.2 Overview of the MulVAL Framework

The MulVAL framework is an open-source tool for generating and analyzing logical attack graphs [58]. MulVAL has three main benefits in comparison to previously suggested tools:

(i) **Automation.** MulVAL models the interactions of software vulnerabilities with system and network configurations, while *automatically* extracting information from formal vulnerability databases (e.g., the National Vulnerability Database [2]) and network scanning tools (e.g., Nessus [1]).

(ii) **Efficiency.** The complexity of other existing tools (e.g., model checking tools) when enumerating all possible attack paths is exponential in the size of the input [58]. In contrast, MulVAl enumerates all possible attack paths in polynomial time.

(iii) **Extendibility.** MulVAL is an open-source framework, therefore it can be extended with new features. Furthermore, MulVAL's expressive capability can be extended with new attack scenarios.

Consequently, multiple risk assessment and countermeasure planning algorithms use MulVAL as a method for estimating risk [4, 11, 30, 78].

Within MulVAL, interaction rules and predicates are written in Datalog, which is a subset of the Prolog programming language. The Datalog language includes three main entities, namely, the variables, constant values, and predicates. Variables always start with an uppercase letter; they can be instantiated with any value during evaluation. Constant values always start with a lowercase letter, and they are set during the formulation of the rule. Predicates are an atomic formula of the form: $p(t_1, \ldots, t_k)$, where each argument t_i can be either a variable or a constant value.

In Listing 1, we present a simple interaction rule (written in Datalog) that represents a general attack technique—the remote exploit of a privilege escalation vulnerability in a program—which leads to the attacker's potential privilege to execute arbitrary code on the host running the program.

```
1 execCode2(Attacker,Host,Privilege) :-
2  networkService5(Host,Program,Protocol,Port,Privilege),
3  vulExists5(Host,VulID,Program,remote,pEscalation),
4  netAccess4(Attacker,Host,Protocol,Port).
```

Listing 1. An example to a simple interaction rule.

This generic interaction rule specifies the preconditions and consequence for this attack. Specifically, if a *Program* is running in a *Privilege* on a *Host* as a network service listening

on *Protocol* and *Port* (the first precondition); and this *Program* contains a remotely exploitable (*remote*) vulnerability (*VulID*) whose impact is privilege escalation (the second precondition); and the *Attacker* can access the service running on the *Host* through the network on *Protocol* and *Port* (the third precondition), then the attacker can execute arbitrary code on the *Host* to gain *Privilege* (attack consequence).

6.3 Limitations of MulVAL

While MulVAL is a very powerful tool, like all risk assessment frameworks it requires high maintenance. Specifically, to cope with new attack trends and emerging threats, risk assessment frameworks must be updated regularly with up-to-date information on new vulnerabilities and attack techniques. In MulVAL, attack techniques are represented using predicates and interaction rules, and the process of formulating new predicates and interaction rules for representing a new attack technique is referred to as *attack modeling*. Consequently, to incorporate attacks on ML production systems, a wide range of new attack techniques (such as AML) should be considered and modeled.

Previous attempts to extend MulVAL have focused on network attacks [3, 7, 27, 47, 49, 79], data vulnerability, cloud security [24], and IoT environments [4]. These extensions, however, do not consider attacks on ML systems. For instance, in a *poisoning attack*, the attacker must have the ability to manipulate the model's training data, however, the relation between a model and its training set is a property that is specific to ML applications. This property is not considered in the previous extensions to MulVAL. Therefore, existing methods cannot be used to quantify the risk of an enterprise network that includes ML systems. To the best of our knowledge, a threat analysis framework that considers cyber attacks on ML systems has yet to be presented.

In this work, we follow the methodology presented in Reference [37] and extend MulVAL to represent cyberattacks on ML systems. The proposed extension enhances the power of attack-graph-based risk assessment by introducing paradigms that have not been modeled before, thus providing security practitioners with a tactical tool for evaluating the impact and quantifying the risk of a cyberattack targeting ML systems.

6.4 The Proposed Extension

In this section, we present a MulVAL extension that considers attacks on ML systems. Due to space limitations, we are unable to present all of the predicates and interaction rules included in the proposed extension. Instead, we provide guidelines for creating these predicates and demonstrate the guidelines using examples. The extension (including all of the predicates and interaction rules) is available **here**.

Modeling pipeline components. To incorporate attacks on ML systems within the MulVAL attack graph analysis tool, we must first model the basic components of an ML system. This is done by creating new *primitive* predicates for all of the target assets described in Section 3.1. For example, the most basic components of any ML system are the learning algorithm and the ML model, and in Listing 2, we demonstrate how we model these components using the Datalog language. Specifically, *algorithm*3 (line 1) is a *primitive* predicate that specifies a learning algorithm. This predicate includes the following two characteristics: *AlgorithmID* and *AlgorithmHost*, which indicate the path to the source code of the algorithm (*AlgortihmID*) and the IP address of the host storing that source code (*AlgorithmHost*), respectively; *model*6 (line 2) is a *primitive* predicate that specifies an ML model (*ModelID*) that is assigned to a specific pipeline (*PipelineID*) and is created by applying the specific learning algorithm (*AlgorithmID*) on specific training data (*TrainingDataID*).

Following the definition of a learning algorithm and ML model, we define the attributes of such models. For example, *vulModel*5 (line 3) specifies whether a certain ML model (*ModelID*),

trained by a specific learning algorithm (*AlgorithmID*), has a particular type (*Type*) of security vulnerability; and *publicModel* (line 4) specifies whether a certain model (*ModelID*) is publicly available.

Modeling data types. ML production pipelines have multiple data types. In our model, we consider the following nine types of data: raw, feature, label, training, validation, evaluation, prediction, and label data. For each data type, we created a *primitive* predicate that includes all relevant characteristics of that type of data.

For example, in Listing 2, we demonstrate how we model training data (line 5). Specifically, the *trainingData*3 *primitive* predicate includes the following four characteristics: *pipelineID*, *ModelID*, *DataID*, and *Host*, which specify the pipeline and model that utilizes this data, a unique identifier of that data (e.g., the path of this data), and the location of that data (e.g., the IP of the host storing the data).

```
1  algorithm3(AlgorithmID,AlgorithmHost).
2  model6(PipelineID,AlgorithmID,ModelID,ModelHost,TrainingDataID,DataHost).
3  vulModel5(PipelineID,AlgorithmID,ModelID,ModelHost,Type).
4  publicModel1(ModelID).
5  trainingData2(PiplineID,ModelID,DataID,Host).
```

Listing 2. ML components and data types.

Modeling data operations. ML pipelines include multiple data operations, such as data normalization, data aggregation, and feature extraction. In our modeling, we refer to these operations as data transformation jobs (line 1 in Listing 3). A data transformation job (*DataTransformationJob*) is a *primitive* predicate that represents a task (identified by *JobID*), which takes data in one format (*InputDataID*) and transforms it into another format (*TransformedDataID*) by applying some transformation (e.g., feature normalization). Since the hosts running the task are not necessarily the same as the hosts storing the input/output data, in our modeling, the data transformation job also includes variables that specify the host running the job (*JobHost*), the host storing the input data (*InputDataHost*), and the host storing the transformed data (*TransformedDataHost*).

In addition, since in production pipelines data operations are commonly performed on computing clusters, we extend our modeling to support the use of computing clusters to run data transformation jobs (lines 2–8). We start by modeling the relationship between the three basic cluster components: master, edge (gateway host), and worker nodes. Specifically, the master node is responsible for task management and resource scheduling among the worker nodes that store data and execute tasks on the data. The edge nodes are used for getting data in and out of the cluster and for data processing job submissions. In our modeling, this relationship is represented using the *clusterWorker*3 predicate (line 2), which indicates that a certain host (*WorkerHost*) is part of a computing cluster for which *ClusterGatewayHost* and *ClusterMasterHost* are, respectively, the gateway and master hosts.

We further extend the definitions of raw data and data transformation jobs to support the dynamics of a computing cluster. Specifically, we create derivation rules for the *rawData*2 and *dataTransformationJob*6 predicates, which propagate raw data and data transformation jobs from the master node to the worker nodes (lines 3–8).

Modeling monitoring services. To maintain the quality of the deployed model and trigger model retraining or data collection, if necessary, model performance should be monitored continuously. To model monitoring operations, we create the *performanceMonitoring* primitive predicate (line 1, Listing 4). We assume that performance monitoring is performed using some dataset (*DataID*) on a specific worker (*Host*) and for a specific pipeline (*PipelineID*). We further assume that the

```
dataTransformationJob6(JobID,JobHost,InputDataID,InputDataHost,TransformedDataID,TransformedDataHost).
clusterWorker3(WorkerHost,ClusterGatewayHost,ClusterMasterHost)
rawData2(DataID,WorkerHost):-
  rawData2(DataID,ClusterGatewayHost),
  clusterWorker3(WorkerHost,ClusterGatewayHost,ClusterMasterHost)
dataTransformationJob6(JobID,WorkerHost,InputDataID,WorkerHost,TransformedDataID,TransformedDataHost):-
  dataTransformationJob6(JobID,ClusterGatewayHost,InputDataID,ClusterGatewayHost,TransformedDataID,
       TransformedDataHost),
  clusterWorker3(WorkerHost,ClusterGatewayHost,ClusterMasterHost)
```

Listing 3. Data propagation in big data architecture.

performance monitoring job is controlled by some program (*Program*). When a decrease in the model's performance is detected, model retraining and/or the collection of new data can be executed. We refer to these as *triggerModelTraining*4 and *triggerDataExtraction*4, respectively (lines 2–3). To describe the relationship between each ML job and the monitoring job, the worker node monitoring the model performance and starting retraining or data collection is specified (*MonitoringHost*).

```
performanceMonitoring5(PiplineID,ModelID,DataID,Host,Program).
triggerModelTraining4(PiplineID,ModelID,MonitoringHost,TrainingHost).
triggerDataExtraction4(PiplineID,ModelID,MonitoringHost,DataTransformationJobHost).
```

Listing 4. Performance monitoring.

Modeling attacker's access capabilities. We create primitive predicates for all of the access capabilities described in Section 3.2, where each predicate includes the following four characteristics: *Principal*, *PipelineID*, *ModelID*, and *AccessLevel*, which outline the principal that acquires the capability, the affected pipeline, the affected model, and the access level (e.g., read/write). Examples of these predicates for model access and query access are presented in Listing 5 (lines 1–2).

```
modelAccess4(Principal,PipelineID,ModelID,AccessLevel).
queryAccess4(Principal,PipelineID,ModelID,AccessType).
modelAccess4(Principal,PipelineID,ModelID,read):-
  malicious_p1(Principal),
  model6(PipelineID,AlgorithmID,ModelID,ModelHost,TrainingDataID,DataHost),
  accessFile4(Principal,ModelHost,read,ModelID).
queryAccess3(Principal,PipelineID,ModelID,Type):-
  malicious1(Principal),
  predictionService8(PipelineID,ModelID,ServingApiID,PredictionServiceHost,Program,Prot,Port,Type),
  netAccess5(Principal,_,PredictionServiceHost,Prot,Port),
  aclH6(PredictionServiceHost,_,_,PredictionServiceHost,Prot,Port).
```

Listing 5. Attacker's access capabilities.

We further create derivation rules that can be used to assess the different access capabilities from the configuration (state) of the target environment. For example, we create a derivation rule for the *modelAccess*4 predicate, for cases in which a malicious entity can access the model file (lines 3–6). This derivation rule includes the following three preconditions: *malicious*1, which indicates that *Principal* is a malicious entity; *model*6, which specifies that the ML model is stored in path *ModelID* on host *ModelHost*; and *accessFile*4, which indicates that the malicious entity (*Principal*) can access the model file stored in path *ModelID* on host *Host*.

Modeling attacker's knowledge. Based on the threat model described in Section 3.2.2, we create primitive predicates for all of types of information available to the attacker. Each predicate includes the following three characteristics: *Principal*, *PipelineID*, and *ModelID*, which describe the principal that acquires that knowledge on the target system, pipeline, and model. An example of the predicate created for model knowledge is presented in Listing 6 (line 1).

We further create derivation rules that can be used to assess system knowledge from the state of the target environment. For instance, we create derivation rules for the *modelKnowledge*3 predicate, for cases in which an attacker acquires model knowledge through access to the model file (lines 2–5 in Listing 6). This derivation rule includes the following three preconditions: *malicious*1, which indicates that *Principal* is a malicious entity; *model*6, which specifies that the ML model is stored in path *ModelID* on host *ModelHost*; and *modelAccess*4, which indicates that the malicious entity (*Principal*) can access the model file. Another example is an attacker that acquires model knowledge through access to public data sources (lines 6–9 in Listing 6).

```
1  modelKnowledge3(Principal,PipelineID,ModelID).
2  modelKnowledge3(Principal,PipelineID,ModelID):-
3    malicious1(Principal),
4    model6(PipelineID,AlgorithmID,ModelID,ModelHost,TrainingDataID,DataHost),
5    modelAccess4(Principal,PipelineID,ModelID,_).
6  modelKnowledge3(Principal,PipelineID,ModelID):-
7    malicious1(Principal),
8    model6(PipelineID,AlgorithmID,ModelID,ModelHost,TrainingDataID,DataHost),
9    publicModel_p1(ModelID).
```

Listing 6. Attacker's knowledge.

6.5 Threat Categories and Attack Techniques

In the subsections above, we modeled the basic components of ML systems, the dynamics of such systems, and the various threat models. In this section, we use this modeling to model the different attack techniques and map them to the relevant threat categories. Specifically, we create derivation rules for each threat category, where each derivation rule represents an attack technique that can be used to materialize the threat and includes the preconditions for executing the attack.

In Listing 7, we provide two examples for materializing the evasion threat ($T1$): a boundary-based black-box decision-based evasion attack ([$AT3$], lines 1–7) and a transferability-based black-box poisoning attack ([$AT7$], lines 7–17). Specifically, the derivation rule for a boundary-based, black-box decision-based evasion attack includes the following five preconditions: *malicious*1, which indicates that *Principal* is a malicious entity; *model*6, which indicates the existence of an ML model; *vulModel*5, which indicates the existence of an *evasion vulnerability* in the ML model; *queryAccess*5, which specifies that this attack requires query access to the model; and *taskKnowledge*, which indicates that the attacker must know the learning task to successfully execute the attack.

```
1  evasionAttack4(Principal,PiplineID,ModelID,tampering):-
2    malicious1(Principal),
3    model7(PipelineID,AlgorithmID,ModelID,ModelHost,TrainingDataID,LabelsDataID,ValidationDataID),
4    vulModel5(PipelineID,AlgorithmID,ModelID,ModelHost,evasionVulnerability),
5    queryAccess5(Principal,PipelineID,ModelID,decision,full),
6    taskKnowledge4(Principal,PipelineID,ModelID,KnowledgeLevel)
7  evasionAttack4(Principal,PipelineID,ModelID,tampering):-
8    malicious1(Principal),
9    model7(PipelineID,AlgorithmID,ModelID,ModelHost,TrainingDataID,LabelsDataID,ValidationDataID),
10   vulModel5(PipelineID,AlgorithmID,ModelID,ModelHost,transferabilityVulnerability),
11   taskKnowledge4(Principal,PipelineID,ModelID,full)
12   surrogateDataAccess4(Principal,PipelineID,ModelID,full),
13   vulModel5(PipelineID,AlgorithmID,ModelID,ModelHost,poisoningVulnerability),
14   trainingDataAccess6(Principal,PipelineID,ModelID,TrainingDataID,write,limited),
15   labeledDataAccess6(Principal,PipelineID,ModelID,LabeledDataID,write,limited),
16   modelRetrainedContinuously2(PipelineID, ModelID),
17   queryAccess5(Principal,PipleineID,ModelID,decision,limited),
```

Listing 7. Threat categories and attack techniques.

The derivation rule for a transferability-based black-box poisoning attack (which can also materialize the evasion threat) includes 10 preconditions. The first 5 preconditions (lines 8–12)

indicate that the adversary can generate a surrogate model and use that model to create poisoning examples that are transferable to the target model, specifically: *malicious*1, which indicates that *Principal* is a malicious entity; *model*6, which indicates the existence of an ML model; *vulModel*5, which indicates the existence of a *transferability vulnerability* in the ML model; *taskKnowledge*, which indicates that the attacker must know the learning task to successfully execute the attack; and *surogateDataAccess*4, which specifies that the adversary has access to a surrogate dataset with similar attributes and statistical properties. The next 4 preconditions (lines 13–17) indicate that the adversary can inject the generated poisoning examples into the training set of the target model, specifically: *vulModel*5, which indicates the existence of a *poisoning vulnerability* in the target ML model; *trainingDataAccess*6 and *labeledDataAccess*6, which specify that the adversary (denoted by *Principal*) can write to the model's training dataset; and *modelRetrainedContinously*2, which indicates that the target model will eventually be trained on the poisoned training data. The last precondition (line 17) indicates that the adversary has limited query access to the target model. This precondition is important so the attacker can send the crafted samples to the model for classification.

6.6 Risk Assessment Using Attack Graphs and the CMLVSS

In this section, we estimate the risk of each attack path using the proposed AML attack scoring system. Our method is based on the risk calculation method presented in Reference [78], which we modify for the AML attack domain. Specifically, the risk equation of a node a is defined as follows:

$$Risk(a) = I_a \cdot LH(a), \tag{1}$$

where I_a is the *impact* of exploiting the compromised asset represented by node a, and LH is the likelihood of the attacker to reach that asset. Note, for AML threats, the impact is defined based on the attack impact metric from CMLVSS.

The likelihood equations (i.e., $LH(a)$) are computed for each node a in the attack graph and are defined recursively by Equations (3), (4), and (5). Specifically, if node a represents a traditional vulnerability, then $LH(a)$ is calculated according to the exploitability metrics of the CVSS v.2 base score, which is commonly used to assess security risk. Specifically, we assign the probability of vulnerability exploitation based on the CVSS v.2 access complexity metric, as specified in Equation (2):

$$LH(a \in TV) = \begin{cases} 0.35, & AC = High \\ 0.61, & AC = Medium \\ 0.71, & AC = Low \end{cases}, \tag{2}$$

where TV denotes the traditional vulnerability, and AC denotes the CVSS v.2 access complexity metric. We focus only on this metric, because it represents information that cannot be expressed by the other attack graph nodes, unlike the other exploitability metrics (i.e., access vector and authentication).

However, if node a represents an AML vulnerability, then $LH(a)$ is calculated according to the attack performance metric from CMLVSS. Specifically, we assign the probability of AML vulnerability exploitation, as specified in Equation (3):

$$LH(a \in TV) = \begin{cases} 0.35, & AP = High \\ 0.61, & AP = Medium \\ 0.71, & AP = Low \end{cases}. \tag{3}$$

Note that if node a does not represent a vulnerability, then $LH(a) = 1$ (i.e., any other fact can be materialized in any situation).

The following equations describe the likelihood calculation of *AND* and *OR* nodes, respectively. Note that for an *OR* node to be materialized, it is sufficient if one of its parents is materialized. Therefore, to consider each parent only once in the likelihood computation, we use the inclusion-exclusion principle.

$$LH(AND) = \prod_{a \in AND_{in}} LH(a) \tag{4}$$

$$\begin{aligned} LH(OR) = \Bigg(& \sum_{i=1}^{R} LH(a_i) \\ & - \sum_{i_1 < i_2} LH(a_{i_1}) \cdot LH(a_{i_2}) \\ & + \cdots + (-1)^R \cdot \sum_{i_1 < \cdots < i_{R-1}} LH(a_{i_1}) \cdot \ldots \cdot LH(a_{i_{R-1}}) \\ & + (-1)^{R+1} \cdot \prod_{i=1}^{R} LH(a_i) \Bigg) \end{aligned} \tag{5}$$

$$where\ R = |OR_{in}|\ and\ OR_{in} = \{a_1, a_2, \ldots, a_R\}.$$

Algorithm 1 describes the process of building the likelihood equations for each node in a given attack graph (denoted by *AG*). It maintains two sets of nodes: a *processed* set that contains all of the nodes that have already been processed (i.e., their equation has been constructed) and an *unprocessed* set that contains the remaining nodes. In each iteration, a node that can be processed (i.e., all of its parents have been processed) is chosen, and its representative equation is constructed until all nodes have been processed.

ALGORITHM 1: Build likelihood equations.

```
1:  processed ← {(a, LH(a)) | a ∈ AG_LEAF}
2:  unprocessed ← AG \ AG_LEAF
3:  while !unprocessed.isEmpty() do
4:      while !unprocessed.hasProcessableNode() do
5:          a ← unprocessed.pop()
6:          a.updateMA()
7:          if ∀k ∈ a_in, k ∈ processed then
8:              processed ← processed ∪ {(a, LH(a))}
9:          else
10:             unprocessed.push(a)
11:         end if
12:     end while
13: end while
```

7 DEMONSTRATION

In this section, we demonstrate the impact of attack graph analysis using the proposed extension. We begin by describing a simplified (and vulnerable) target environment that includes ML components. Then, we use the proposed extension to generate an attack graph for the target environment and discuss the results.

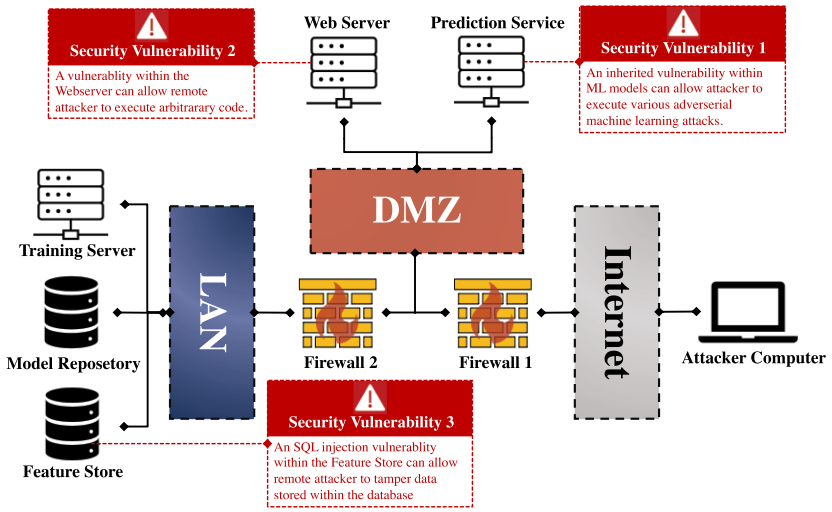

Fig. 9. The evaluation framework.

7.1 Target Environment

The target environment includes the following eight components (see Figure 9): web server, prediction service, training server, model repository, feature store, client application, firewall 1, and firewall 2.

(i) **Web server.** Thiscomponent is responsible for hosting the enterprise website. The web server is located in the DMZ and is reachable from the Internet. The web server has access to the feature store, which is located on the LAN. The web server runs on a vulnerable version of the Apache web server, which can allow a remote attacker to execute arbitrary code.

(ii) **Prediction service.** This component is responsible for serving the ML applications running on the client application. The prediction service is located in the DMZ and is reachable from the Internet on TCP port 80. The prediction service has access to the feature store (which is located on the LAN) on port 3036. The ML model, which is encapsulated by the prediction service, is vulnerable to evasion and poisoning attacks.

(iii) **Training server.** This component is responsible for producing the ML model by running the learning algorithms on the training data (from the feature store). To cope with concept drifts, an ML model trained on fresh data (which exists in the feature store) is deployed to the serving server every hour (for simplicity, we do not include A/B testing in this environment). The training server is located on the LAN.

(iv) **Model repository.** This component is responsible for storing trained models, along with their performance and versioning, and other configuration information. The model repository is located on the LAN.

(v) **Feature store.** This component is responsible for storing and logging features for ML tasks. The feature store is located on the LAN and listens for incoming requests on TCP port 3036. The feature store is vulnerable to SQL injection, which can allow a remote attacker to tamper with data stored in the feature store (e.g., manipulating the data used to train the ML model).

(vi) **Attacker computer (client application).** This component is responsible for querying the prediction service. The client application is located on the Internet, and we assume that it can be compromised by an attacker that is masquerading as a legitimate client.

(vii) **Firewall 1.** This component is responsible for monitoring and controlling the incoming and outgoing network traffic transmitted from/to the Internet. Firewall 1 allows the incoming TCP connection from the Internet to the web server on Ports 80 and 443 and the incoming TCP connection from the Internet to the prediction service on port 8080. In addition, Firewall 1 allows the outgoing TCP connection from the LAN to the Internet on port 80. All other types of communication are blocked.

(viii) **Firewall 2.** This component is responsible for monitoring and controlling the incoming and outgoing network traffic transmitted from/to the Internet or DMZ. Firewall 2 allows the incoming TCP connection from the web server to the feature store on port 3306 and the incoming TCP connection from the prediction service to the feature store on port 3306. All other types of communication are blocked.

7.2 Attack Graph Analysis

We now use the proposed extension to generate an attack graph for the target environment. To produce an attack graph that is not too large (which will make the demonstration difficult to follow), we made the following two decisions: First, we limit the attack graph analysis framework to evaluating the materialization of just the first threat *[T1]*, i.e., evading an ML system. Second, we limit the adversarial attack techniques considered in the assessment to boundary-based black-box decision-based evasion attacks *[AT3]* and transferability-based targeted black-box poisoning attacks *[AT7]*. The input and interaction rule files are provided as supplementary material. The resulting attack graph is presented in Figure 10.

As can be seen, the attack graph generated includes the following two main attack paths:

Attack Path 1. This attack path (in red) materializes the evasion threat by implementing a boundary-based black-box decision-based evasion attack. This attack path, which includes three phases, is based solely on AML. In the first phase, the attacker acquires network access to the prediction service located in the DMZ. This is possible, because the prediction service is reachable from the Internet on port 3036. In the second phase, the attacker acquires decision-based query access in the ML pipeline. This is made possible, because the prediction service exposes a serving API to the client application, which is controlled by the attacker. In the third phase, the attacker executes a boundary-based black-box decision-based evasion attack. Note that this attack path is based solely on AML attack techniques.

To quantify the risk of this attack path, we follow the procedure described in Section 6.6. Specifically, in attack path 1, the AML vulnerability is exploited by executing a black-box evasion attack. According to CMLVSS, the performance of this attack is classified as MEDIUM, which results in a probability of 0.61. In addition, the impact of this attack is classified as tampering, which translates to an impact value of 0.385. Thus, according to Algorithm 1, the risk of this path is equal to 0.234.

Attack Path 2. This attack path (in blue) materializes the evasion threat by implementing a transferability-based black-box poisoning attack. This attack path, which includes five phases, combines the exploitation of traditional cybersecurity vulnerabilities as well as AML. In the first phase, the attacker acquires network access to the web server located in the DMZ. This is possible, because the web server is reachable from the Internet on port 80. In the second phase, the attacker exploits a remote code execution vulnerability that exists on the Apache web server, which results in the attacker obtaining remote control of the web server. In the third phase, the attacker leverages this remote control and acquires network access to the feature store located on the LAN. This is possible, because the feature store listens for incoming requests on TCP port 3036, and firewall 2 allows incoming communication from the web server to the feature store on TCP port 3036. In

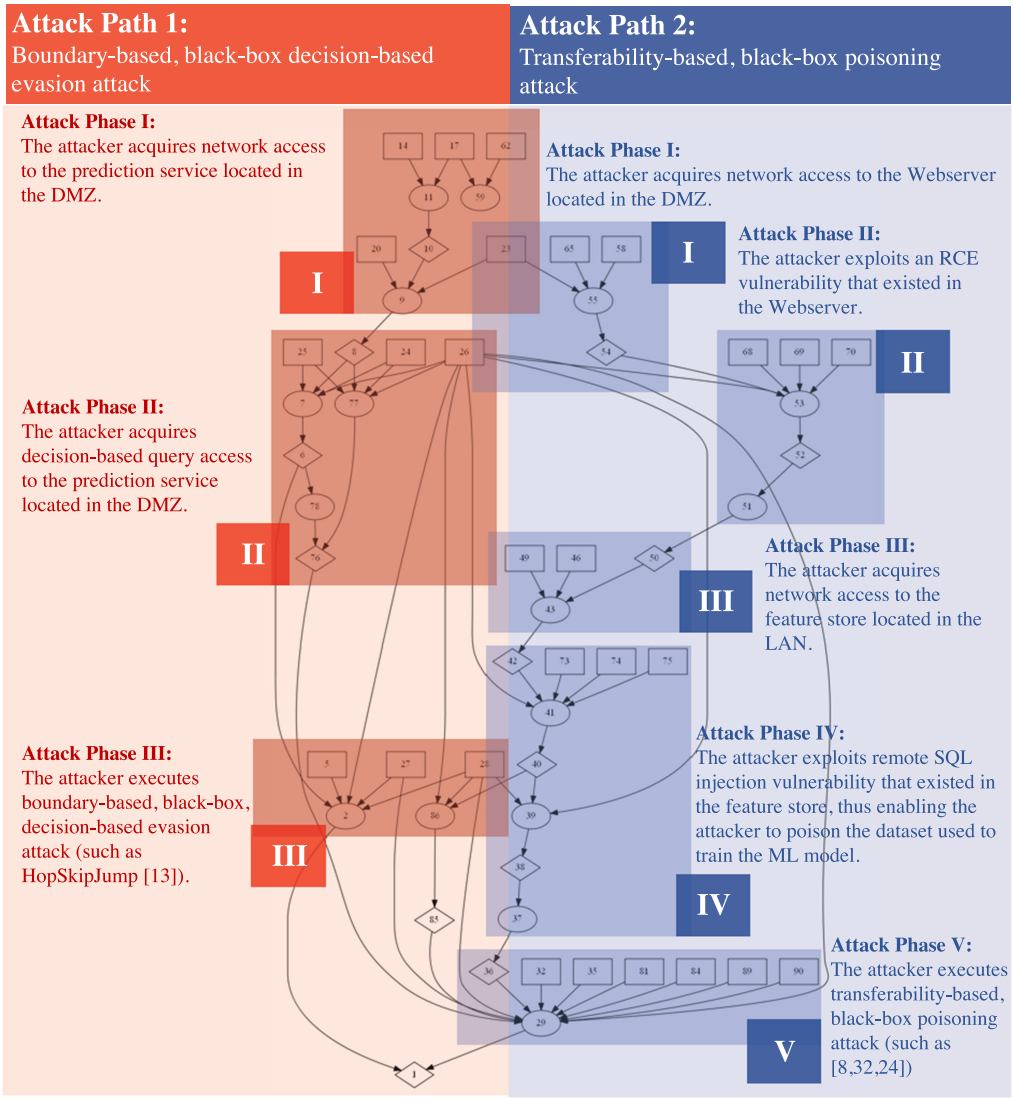

Fig. 10. The attack graph generated by using the proposed extension.

the fourth phase, the attacker exploits an SQL injection vulnerability in the feature store, which could give the attacker the ability to manipulate the data stored in the feature store. Since this data is used by the ML pipeline to train the learning algorithm, by manipulating the data stored in the feature store, the attacker can poison the ML model's training dataset. In the fifth phase, the attacker executes a transferability-based black-box poisoning attack, which can materialize the evasion threat.

To quantify the risk of this attack path, we follow the procedure described in Section 6.6. Specifically, in attack path 2, the attacker exploits the following three vulnerabilities: (a) a traditional vulnerability on the web server; according to CVSS 2.1, this vulnerability can be exploited with LOW complexity, which translated to a probability of 0.71; (b) a traditional vulnerability on the SQL server; according to CVSS 2.1, this vulnerability can be exploited with MEDIUM complexity,

which translates to a probability of 0.61; (c) an AML vulnerability, which is exploited by executing a transferability-based poisoning attack. According to the CMLVSS, the performance of this attack is classified as LOW, which results in a probability of 0.35. In addition, the impact of this attack is classified as tampering, which translates to an impact value of 0.385. Thus, according to Algorithm 1, the risk of this path is equal to 0.05.

The algorithm suggests eliminating attack path 1 first. The main reason for this is because attack path 2 requires the exploitation of additional vulnerabilities to achieve the preconditions required for executing a poisoning attack. Note that in these attack scenarios the analysis performed by CMLVSS and MulVAL (using the proposed extension) yields similar insights.

8 RELATED WORK

Previous studies proposed taxonomies for the security and privacy of ML systems. For example, Papernot et al. [65] analyzed the security of ML systems while considering the adversary's capabilities (e.g., white-box or black-box, different threats (e.g., an attack during inference phase or training phase)) and the adversary's goal (integrity, privacy, or availability; targeted vs. indiscriminate). Huang et al. [35] proposed a taxonomy for classifying attacks against online ML algorithms and demonstrated attacks in two use cases (spam detection and network anomaly detection). The authors also considered how an adversary attacks an ML system and discussed possible defenses and countermeasures.

In their work, Biggio et al. [10] provided a comprehensive overview of the evolution of the adversarial learning research domain over a 10-year period, specifically for computer vision and cybersecurity tasks. In their attacker model, the authors distinguished between the attacker's knowledge and capabilities.

Kumar et al. [43] surveyed 28 different organizations. The authors found that industry practitioners are not equipped with the proper knowledge and tools to protect, detect, and respond to attacks on their ML systems. In addition, based on the questionnaires, the authors enumerated the gaps and suggested possible approaches for improving the security of ML systems during the development lifecycle.

Spring et al. [76] discussed the idea of managing vulnerabilities in ML systems according to the CVE. The authors only considered the ML algorithms and model objects.

Lin and Biggio [46] found that there is a gap between AML attacks in laboratories and the real-world, however, they focused on whether the attack could successfully be transferred from a research environment to a production environment.

The studies mentioned above are different from ours in the following ways: First, none of the works above considered the differences and unique properties of production ML-based systems, which, as presented in this work, introduce additional attack vectors. Second, none of the studies suggested a practical method for quantifying the risk of an ML-based attack on the systems. Finally, we provide a practical threat analysis of ML-based systems that presents a comprehensive list of attack techniques and the ability to use these attack techniques in an attack graph-based threat analysis process that combines both cyber and AML threats.

9 CONCLUSIONS

In this article, we performed a comprehensive threat analysis of ML production systems, noting the differences between ML research and production pipelines. First, We identified the primary assets of ML production systems and discussed the threats to these assets, and then we presented an extensive threat model, which considers the attacker's access capabilities, knowledge, and goals; the threat model integrates multiple threat models described in previous works. Various attack techniques against ML systems were reviewed, and for each attack technique, we provided a brief

description of the attack method, identified the assets targeted by the adversary, and mentioned the relevant threat model. In addition to conducting a comprehensive threat analysis of ML production systems, we developed an extension to the MulVAL attack-graph analysis framework that incorporates cyberattacks on ML production systems. The proposed extension will enable security practitioners to apply attack graph analysis methods in environments that include ML components, thus providing security experts with a practical tool for evaluating the impact of a cyberattack targeting an ML production system and quantifying the risk of the attack.

REFERENCES

[1] 2022. Nessus security scanner. Retrieved from http://www.nessus.org.

[2] 2022. NVD national vulnerability database. Retrieved from http://www.nvd.nist.gov.

[3] Jaime C. Acosta, Edgar Padilla, and John Homer. 2016. Augmenting attack graphs to represent data link and network layer vulnerabilities. In *Proceedings of the IEEE Military Communications Conference*. IEEE, 1010–1015.

[4] Noga Agmon, Asaf Shabtai, and Rami Puzis. 2019. Deployment optimization of IoT devices through attack graph analysis. In *Proceedings of the 12th Conference on Security and Privacy in Wireless and Mobile Networks*. 192–202.

[5] Giuseppe Ateniese, Luigi V. Mancini, Angelo Spognardi, Antonio Villani, Domenico Vitali, and Giovanni Felici. 2015. Hacking smart machines with smarter ones: How to extract meaningful data from machine learning classifiers. *Int. J. Secur. Netw.* 10, 3 (2015), 137–150.

[6] John O. Awoyemi, Adebayo O. Adetunmbi, and Samuel A. Oluwadare. 2017. Credit card fraud detection using machine learning techniques: A comparative analysis. In *Proceedings of the International Conference on Computing Networking and Informatics (ICCNI)*. IEEE, 1–9.

[7] Eugen Bacic, Michael Froh, and Glen Henderson. 2006. *Mulval Extensions for Dynamic Asset Protection.* Technical Report. Cinnabar Networks, Inc.

[8] Denis Baylor, Eric Breck, Heng-Tze Cheng, Noah Fiedel, Chuan Yu Foo, Zakaria Haque, Salem Haykal, Mustafa Ispir, Vihan Jain, Levent Koc, et al. 2017. TFX: A tensorflow-based production-scale machine learning platform. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1387–1395.

[9] Battista Biggio, Blaine Nelson, and Pavel Laskov. 2012. Poisoning attacks against support vector machines. *arXiv preprint arXiv:1206.6389* (2012).

[10] Battista Biggio and Fabio Roli. 2018. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recogn.* 84 (2018), 317–331.

[11] Hodaya Binyamini, Ron Bitton, Masaki Inokuchi, Tomohiko Yagyu, Yuval Elovici, and Asaf Shabtai. 2020. An automated, end-to-end framework for modeling attacks from vulnerability descriptions. *arXiv preprint arXiv:2008.04377* (2020).

[12] Hodaya Binyamini, Ron Bitton, Masaki Inokuchi, Tomohiko Yagyu, Yuval Elovici, and Asaf Shabtai. 2021. A framework for modeling cyber attack techniques from security vulnerability descriptions. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2574–2583. DOI: https://doi.org/10.1145/3447548.3467159

[13] Ron Bitton and Asaf Shabtai. 2021. A machine learning-based intrusion detection system for securing remote desktop connections to electronic flight bag servers. *IEEE Trans. Depend. Secur. Comput.* 18, 3 (2021), 1164–1181. DOI: https://doi.org/10.1109/TDSC.2019.2914035

[14] Wieland Brendel, Jonas Rauber, and Matthias Bethge. 2017. Decision-based adversarial attacks: Reliable attacks against black-box machine learning models. *arXiv preprint arXiv:1712.04248* (2017).

[15] Nicholas Carlini and David Wagner. 2017. Adversarial examples are not easily detected: Bypassing ten detection methods. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*. 3–14.

[16] Varun Chandrasekaran, Kamalika Chaudhuri, Irene Giacomelli, Somesh Jha, and Songbai Yan. 2020. Exploring connections between active learning and model extraction. In *29th USENIX Security Symposium (USENIX Security'20)*. 1309–1326.

[17] Jianbo Chen, Michael I. Jordan, and Martin J. Wainwright. 2020. HopSkipJumpAttack: A query-efficient decision-based attack. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*. IEEE, 1277–1294.

[18] Pin-Yu Chen, Huan Zhang, Yash Sharma, Jinfeng Yi, and Cho-Jui Hsieh. 2017. Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*. 15–26.

[19] Chun-Jen Chung, Pankaj Khatkar, Tianyi Xing, Jeongkeun Lee, and Dijiang Huang. 2013. NICE: Network intrusion detection and countermeasure selection in virtual network systems. *IEEE Trans. Depend. Secure Comput.* 10, 4 (2013), 198–211.

[20] Francesco Croce and Matthias Hein. 2020. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *Proceedings of the International Conference on Machine Learning*. PMLR, 2206–2216.
[21] Chaitrali S. Dangare and Sulabha S. Apte. 2012. Improved study of heart disease prediction system using data mining classification techniques. *Int. J. Comput. Applic.* 47, 10 (2012), 44–48.
[22] Ürün Dogan, Johann Edelbrunner, and Ioannis Iossifidis. 2011. Autonomous driving: A comparison of machine learning techniques by means of the prediction of lane change behavior. In *Proceedings of the IEEE International Conference on Robotics and Biomimetics*. IEEE, 1837–1843.
[23] Evan Downing. 2019. MLsploit [judges remarks], Georgia Institute of Technology. https://smartech.gatech.edu/handle/1853/61037.
[24] Iman El Mir, El Mehdi Kandoussi, Mohamed Hanini, Abdelkrim Haqiq, and Dong Seong Kim. 2017. A game theoretic approach based virtual machine migration for cloud environment security. *Int. J. Commun. Netw. Inf. Secur.* 9, 3 (2017), 345–357.
[25] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. 2015. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 1322–1333.
[26] Matthew Fredrikson, Eric Lantz, Somesh Jha, Simon Lin, David Page, and Thomas Ristenpart. 2014. Privacy in pharmacogenetics: An end-to-end case study of personalized warfarin dosing. In *Proceedings of the 23rd USENIX Security Symposium (USENIX Security'14)*. 17–32.
[27] Michael John Froh and Glen Henderson. 2009. *MulVAL Extensions II*. Defence R & D Canada-Ottawa.
[28] Karan Ganju, Qi Wang, Wei Yang, Carl A. Gunter, and Nikita Borisov. 2018. Property inference attacks on fully connected neural networks using permutation invariant representations. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 619–633.
[29] Eduardo A. Gerlein, Martin McGinnity, Ammar Belatreche, and Sonya Coleman. 2016. Evaluating machine learning classification for financial trading: An empirical approach. *Expert Syst. Applic.* 54 (2016), 193–207.
[30] Tom Gonda, Tal Pascal, Rami Puzis, Guy Shani, and Bracha Shapira. 2018. Analysis of attack graph representations for ranking vulnerability fixes. In *Proceedings of the Global Conference on Artificial Intelligence*. 215–228.
[31] Gustavo Gonzalez-Granadillo, Elena Doynikova, Igor Kotenko, and Joaquin Garcia-Alfaro. 2017. Attack graph-based countermeasure selection using a stateful return on investment metric. In *Proceedings of the International Symposium on Foundations and Practice of Security*. Springer, 293–302.
[32] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572* (2014).
[33] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. 2017. BadNets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733* (2017).
[34] Seira Hidano, Takao Murakami, Shuichi Katsumata, Shinsaku Kiyomoto, and Goichiro Hanaoka. 2017. Model inversion attacks for prediction systems: Without knowledge of non-sensitive attributes. In *Proceedings of the 15th Annual Conference on Privacy, Security and Trust (PST)*. IEEE, 115–11509.
[35] Ling Huang, Anthony D. Joseph, Blaine Nelson, Benjamin I. P. Rubinstein, and J. Doug Tygar. 2011. Adversarial machine learning. In *Proceedings of the 4th ACM Workshop on Security and Artificial Intelligence*. 43–58.
[36] Andrew Ilyas, Logan Engstrom, Anish Athalye, and Jessy Lin. 2017. Query-efficient black-box adversarial examples (superceded). *arXiv preprint arXiv:1712.07113* (2017).
[37] Masaki Inokuchi, Yoshinobu Ohta, Shunichi Kinoshita, Tomohiko Yagyu, Orly Stan, Ron Bitton, Yuval Elovici, and Asaf Shabtai. 2019. Design procedure of knowledge base for practical attack graph generation. In *Proceedings of the ACM Asia Conference on Computer and Communications Security*. 594–601.
[38] Matthew Jagielski, Alina Oprea, Battista Biggio, Chang Liu, Cristina Nita-Rotaru, and Bo Li. 2018. Manipulating machine learning: Poisoning attacks and countermeasures for regression learning. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*. IEEE, 19–35.
[39] Mika Juuti, Sebastian Szyller, Samuel Marchal, and N. Asokan. 2019. PRADA: Protecting against DNN model stealing attacks. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 512–527.
[40] Phongphun Kijsanayothin and Rattikorn Hewett. 2010. Analytical approach to attack graph analysis for network security. In *Proceedings of the International Conference on Availability, Reliability and Security*. IEEE, 25–32.
[41] I. Kotenko and E. Doynikova. 2016. Dynamical calculation of security metrics for countermeasure selection in computer networks. In *Proceedings of the 24th Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP)*. 558–565. DOI: https://doi.org/10.1109/PDP.2016.96
[42] Moshe Kravchik, Battista Biggio, and Asaf Shabtai. 2021. Poisoning attacks on cyber attack detectors for industrial control systems. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing*. 116–125.
[43] Ram Shankar Siva Kumar, Magnus Nyström, John Lambert, Andrew Marshall, Mario Goertzel, Andi Comissoneru, Matt Swann, and Sharon Xia. 2020. Adversarial machine learning-industry perspectives. In *Proceedings of the IEEE Security and Privacy Workshops (SPW)*. IEEE, 69–75.

[44] Alexey Kurakin, Ian J. Goodfellow, and Samy Bengio. 2018. Adversarial examples in the physical world. In *Artificial Intelligence Safety and Security*. Chapman and Hall/CRC, 99–112.

[45] Jesse Levinson, Jake Askeland, Jan Becker, Jennifer Dolson, David Held, Soeren Kammel, J. Zico Kolter, Dirk Langer, Oliver Pink, Vaughan Pratt, et al. 2011. Towards fully autonomous driving: Systems and algorithms. In *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 163–168.

[46] Hsiao-Ying Lin and Battista Biggio. 2021. Adversarial machine learning: Attacks from laboratories to the real world. *Computer* 54, 5 (2021), 56–60.

[47] Changwei Liu, Anoop Singhal, and Duminda Wijesekera. 2015. A logic-based network forensic model for evidence analysis. In *Proceedings of the IFIP International Conference on Digital Forensics*. Springer, 129–145.

[48] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. 2017. Trojaning attack on neural networks. (2017). https://docs.lib.purdue.edu/cgi/viewcontent.cgi?article=2782&context=cstech.

[49] Monali Mavani and Krishna Asawa. 2017. Modeling and analyses of IP spoofing attack in 6LoWPAN network. *Comput. Secur.* 70 (2017), 95–110.

[50] Shike Mei and Xiaojin Zhu. 2015. Using machine teaching to identify optimal training-set attacks on machine learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

[51] Marco Melis, Ambra Demontis, Maura Pintor, Angelo Sotgiu, and Battista Biggio. 2019. SecML: A Python library for secure and explainable machine learning. *arXiv preprint arXiv:1912.10013* (2019).

[52] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. 2016. DeepFool: A simple and accurate method to fool deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2574–2582.

[53] Luis Muñoz-González, Battista Biggio, Ambra Demontis, Andrea Paudice, Vasin Wongrassamee, Emil C. Lupu, and Fabio Roli. 2017. Towards poisoning of deep learning algorithms with back-gradient optimization. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*. 27–38.

[54] Asaf Nadler, Avi Aminov, and Asaf Shabtai. 2019. Detection of malicious and low throughput data exfiltration over the DNS protocol. *Comput. Secur.* 80 (2019), 36–53.

[55] Milad Nasr, Reza Shokri, and Amir Houmansadr. 2019. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*. IEEE, 739–753.

[56] Maria-Irina Nicolae, Mathieu Sinn, Minh Ngoc Tran, Beat Buesser, Ambrish Rawat, Martin Wistuba, Valentina Zantedeschi, Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, et al. 2018. Adversarial robustness toolbox v1. 0.0. *arXiv preprint arXiv:1807.01069* (2018).

[57] Seong Joon Oh, Bernt Schiele, and Mario Fritz. 2019. Towards reverse-engineering black-box neural networks. In *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Springer, 121–144.

[58] Xinming Ou, Sudhakar Govindavajhala, and Andrew W. Appel. 2005. MulVAL: A logic-based network security analyzer. In *Proceedings of the USENIX Security Symposium*. 113–128.

[59] Felipe Dias Paiva, Rodrigo Tomás Nogueira Cardoso, Gustavo Peixoto Hanaoka, and Wendel Moreira Duarte. 2019. Decision-making for financial trading: A fusion approach of machine learning and portfolio selection. *Expert Syst. Applic.* 115 (2019), 635–655.

[60] Sellappan Palaniappan and Rafiah Awang. 2008. Intelligent heart disease prediction system using data mining techniques. In *Proceedings of the IEEE/ACS International Conference on Computer Systems and Applications*. IEEE, 108–115.

[61] Nicolas Papernot, Fartash Faghri, Nicholas Carlini, Ian Goodfellow, Reuben Feinman, Alexey Kurakin, Cihang Xie, Yash Sharma, Tom Brown, Aurko Roy, Alexander Matyasko, Vahid Behzadan, Karen Hambardzumyan, Zhishuai Zhang, Yi-Lin Juang, Zhi Li, Ryan Sheatsley, Abhibhav Garg, Jonathan Uesato, Willi Gierke, Yinpeng Dong, David Berthelot, Paul Hendricks, Jonas Rauber, and Rujun Long. 2018. Technical report on the CleverHans v2.1.0 adversarial examples library. *arXiv preprint arXiv:1610.00768* (2018).

[62] Nicolas Papernot, Patrick McDaniel, and Ian Goodfellow. 2016. Transferability in machine learning: From phenomena to black-box attacks using adversarial samples. *arXiv preprint arXiv:1605.07277* (2016).

[63] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. 2017. Practical black-box attacks against machine learning. In *Proceedings of the ACM on Asia Conference on Computer and Communications Security*. 506–519.

[64] Nicolas Papernot, Patrick McDaniel, Somesh Jha, Matt Fredrikson, Z. Berkay Celik, and Ananthram Swami. 2016. The limitations of deep learning in adversarial settings. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 372–387.

[65] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael P. Wellman. 2018. SoK: Security and privacy in machine learning. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 399–414.

[66] Johan Perols. 2011. Financial statement fraud detection: An analysis of statistical and machine learning algorithms. *Audit.: J. Pract. Theor.* 30, 2 (2011), 19–50.
[67] Jonas Rauber, Wieland Brendel, and Matthias Bethge. 2017. Foolbox: A Python toolbox to benchmark the robustness of machine learning models. *arXiv preprint arXiv:1707.04131* (2017).
[68] Thomas L. Saaty. 2008. Decision making with the analytic hierarchy process. *Int. J. Serv. Sci.* 1, 1 (2008), 83–98.
[69] Ganesh Ram Santhanam, Zachary J. Oster, and Samik Basu. 2013. Identifying a preferred countermeasure strategy for attack graphs. In *Proceedings of the 8th Annual Cyber Security and Information Intelligence Research Workshop*. 1–4.
[70] Avishag Shapira, Alon Zolfi, Luca Demetrio, Battista Biggio, and Asaf Shabtai. 2022. Denial-of-service attack on object detection model using universal adversarial perturbation. *CoRR* abs/2205.13618 (2022).
[71] Eitam Sheetrit, Nir Nissim, Denis Klimov, and Yuval Shahar. 2019. Temporal probabilistic profiles for sepsis prediction in the ICU. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2961–2969.
[72] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. 2017. Membership inference attacks against machine learning models. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*. IEEE, 3–18.
[73] Ilia Shumailov, Yiren Zhao, Daniel Bates, Nicolas Papernot, Robert Mullins, and Ross Anderson. 2021. Sponge examples: Energy-latency attacks on neural networks. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 212–231.
[74] Adir Solomon, Michael Michaelshvili, Ron Bitton, Bracha Shapira, Lior Rokach, Rami Puzis, and Asaf Shabtai. 2022. Contextual security awareness: A context-based approach for assessing the security awareness of users. *Knowl. Based Syst.* 246 (2022), 108709. DOI: https://doi.org/10.1016/j.knosys.2022.108709
[75] Congzheng Song and Ananth Raghunathan. 2020. Information leakage in embedding models. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 377–390.
[76] Jonathan M. Spring, April Galyardt, Allen D. Householder, and Nathan VanHoudnos. 2020. On managing vulnerabilities in AI/ML systems. In *Proceedings of the New Security Paradigms Workshop*. 111–126.
[77] Orly Stan, Ron Bitton, Michal Ezrets, Moran Dadon, Masaki Inokuchi, Yoshinobu Ohta, Tomohiko Yagyu, Yuval Elovici, and Asaf Shabtai. 2021. Heuristic approach for countermeasure selection using attack graphs. In *Proceedings of the 34th IEEE Computer Security Foundations Symposium*. IEEE, 1–16. DOI: https://doi.org/10.1109/CSF51468.2021.00003
[78] Orly Stan, Ron Bitton, Michal Ezrets, Moran Dadon, Masaki Inokuchi, Yoshinobu Ohta, Tomohiko Yagyu, Yuval Elovici, and Asaf Shabtai. 2021. Heuristic approach for countermeasure selection using attack graphs. In *Proceedings of the IEEE 34th Computer Security Foundations Symposium (CSF)*. IEEE Computer Society, 63–78.
[79] Orly Stan, Ron Bitton, Michal Ezrets, Moran Dadon, Masaki Inokuchi, Yoshinobu Ohta, Yoshiyuki Yamada, Tomohiko Yagyu, Yuval Elovici, and Asaf Shabtai. 2019. Extending attack graphs to represent cyber-attacks in communication protocols and modern IT networks. *arXiv preprint arXiv:1906.09786* (2019).
[80] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2013. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199* (2013).
[81] Dejan Varmedja, Mirjana Karanovic, Srdjan Sladojevic, Marko Arsenovic, and Andras Anderla. 2019. Credit card fraud detection-machine learning methods. In *Proceedings of the 18th International Symposium INFOTEH-JAHORINA (INFOTEH)*. IEEE, 1–5.
[82] Valentina Viduto, Carsten Maple, Wei Huang, and David López-Peréz. 2012. A novel risk assessment and optimisation model for a multi-objective network security countermeasure selection problem. *Decis. Supp. Syst.* 53, 3 (2012), 599–610.
[83] Huang Xiao, Battista Biggio, Gavin Brown, Giorgio Fumera, Claudia Eckert, and Fabio Roli. 2015. Is feature selection secure against training data poisoning? In *Proceedings of the International Conference on Machine Learning*. PMLR, 1689–1698.
[84] Yang Xin, Lingshuang Kong, Zhi Liu, Yuling Chen, Yanmiao Li, Hongliang Zhu, Mingcheng Gao, Haixia Hou, and Chunhua Wang. 2018. Machine learning and deep learning methods for cybersecurity. *IEEE Access* 6 (2018), 35365–35381.
[85] Ziqi Yang, Jiyi Zhang, Ee-Chien Chang, and Zhenkai Liang. 2019. Neural network inversion in adversarial setting via background knowledge alignment. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 225–240.
[86] Yuanshun Yao, Huiying Li, Haitao Zheng, and Ben Y. Zhao. 2019. Latent backdoor attacks on deep neural networks. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 2041–2055.

Received 4 November 2021; revised 18 July 2022; accepted 3 August 2022